

JADAVPUR UNIVERSITY

188, Raja S.C. Mallick Road, Kolkata – 700032

Department of Computer Science and Engineering, FET.

MASTER OF COMPUTER APPLICATION

1st Year 2nd Semester

DATA STRUCTURE & ALGORITHMS ASSIGNMENT

Name: Snehasish Sarkar

Roll No.: 002510503033

QUESTION 1 : Write a menu-driven program in C to perform the following operations in an integer array allocated memory dynamically. The list may grow or shrink compared to the initial allocation as and when required. Multiple operations may be performed on the existing list without recompiling/re-executing the program. The individual operation should be implemented using a single function.

List of operations:

- a) Create
- b) Display the entire list
- c) Count (total number of elements in the list)
- d) Reverse the list (reversed list should be stored in the same array)
- e) Index of a given element (all indices if there are duplicate entries)
- f) Indexed element
- g) Insert
- h) Delete
- i) Merge
- j) Split
- k) Sort

SORCE CODE :

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <stdbool.h>
#define MAX 20

typedef struct Array{
    int* arr;
    int n;
}Array;

Array create(int n) {
    Array ptr;
    ptr.arr = (int*)malloc(n * sizeof(int));

    if (ptr.arr == NULL) {
        printf("Memory allocation failed.\n");
        ptr.n = 0;
        return ptr;
    }

    ptr.n = n;
    printf("Generating random elements : ");

    for (int i = 0; i < ptr.n; i++) {
        ptr.arr[i] = rand() % 900 + 100;
        printf("%d ", ptr.arr[i]);
    }

    printf("\nArray created successfully.\n");
    return ptr;
}

void display(Array ptr) {
    if (ptr.arr == NULL || ptr.n == 0) {
```

```

        printf("Array is Empty.\n");
        return;
    }

    printf("[ ");
    for (int i = 0; i < ptr.n; i++)
        printf("%d ", ptr.arr[i]);
    printf("]\n");
}

int count(Array ptr) {
    return ptr.n;
}

void reverse(Array *ptr) {
    if (ptr->arr == NULL || ptr->n == 0) {
        printf("Array is Empty.\n");
        return;
    }

    int i = 0, j = ptr->n - 1;

    while (i < j) {
        int temp = ptr->arr[i];
        ptr->arr[i] = ptr->arr[j];
        ptr->arr[j] = temp;
        i++;
        j--;
    }

    printf("Successfully reversed.\n");
}

Array indexOf(Array ptr, int target) {
    Array pos = {NULL, 0};

    if (ptr.arr == NULL || ptr.n == 0) {
        printf("Array is Empty.\n");
        return pos;
    }

    pos.arr = (int *)malloc(ptr.n * sizeof(int));

    if (pos.arr == NULL)
        return pos;

    for (int i = 0; i < ptr.n; i++) {
        if (ptr.arr[i] == target) {
            pos.arr[pos.n] = i;
            pos.n++;
        }
    }
}

```

```

    if (pos.n == 0) {
        free(pos.arr);
        pos.arr = NULL;
    } else {
        int *temp = realloc(pos.arr, pos.n * sizeof(int));
        if (temp != NULL)
            pos.arr = temp;
    }

    return pos;
}

int getElement(Array ptr, int idx) {
    if (ptr.arr == NULL)
        return -1;

    if (idx < 0 || idx >= ptr.n)
        return -2;

    return ptr.arr[idx];
}

Array insert(Array ptr, int val) {
    if (ptr.arr == NULL || ptr.n == 0) {
        ptr.arr = (int *)malloc(sizeof(int));
        if (ptr.arr == NULL)
            return ptr;

        ptr.arr[0] = val;
        ptr.n = 1;
    } else {
        int *temp = realloc(ptr.arr, (ptr.n + 1) * sizeof(int));
        if (temp == NULL) {
            printf("Memory allocation failed.\n");
            return ptr;
        }

        ptr.arr = temp;
        ptr.arr[ptr.n] = val;
        ptr.n++;
    }

    printf("Successfully Inserted.\n");
    return ptr;
}

Array delete(Array ptr, int *val) {
    if (ptr.arr == NULL || ptr.n == 0) {
        printf("Array is Empty.\n");
        return ptr;
    }

    *val = ptr.arr[ptr.n - 1];

```

```

    if (ptr.n == 1) {
        free(ptr.arr);
        ptr.arr = NULL;
        ptr.n = 0;
    } else {
        ptr.n--;
        int *temp = realloc(ptr.arr, ptr.n * sizeof(int));
        if (temp != NULL)
            ptr.arr = temp;
    }

    return ptr;
}

```

```

Array merge(Array *ptr1, Array *ptr2) {
    Array new = {NULL, 0};

    if (ptr1->arr == NULL && ptr2->arr == NULL)
        return new;

    if (ptr1->arr == NULL)
        return *ptr2;

    if (ptr2->arr == NULL)
        return *ptr1;

    new.n = ptr1->n + ptr2->n;
    int *temp = realloc(ptr1->arr, new.n * sizeof(int));

    if (temp == NULL) {
        printf("Merge failed.\n");
        new.n = 0;
        return new;
    }

    new.arr = temp;

    for (int i = 0; i < ptr2->n; i++)
        new.arr[ptr1->n + i] = ptr2->arr[i];

    ptr1->arr = NULL;
    ptr1->n = 0;

    ptr2->arr = NULL;
    ptr2->n = 0;

    printf("Merge Done.\n");
    return new;
}

```

```

void split(Array *ptr1, Array *ptr2, int idx) {
    if (ptr1->arr == NULL) {

```

```

        printf("Array is Empty.\n");
        return;
    }

    if (idx < 0 || idx >= ptr1->n) {
        printf("Invalid Index.\n");
        return;
    }

    ptr2->n = ptr1->n - idx;
    ptr2->arr = (int *)malloc(ptr2->n * sizeof(int));

    if (ptr2->arr == NULL)
        return;

    for (int i = 0; i < ptr2->n; i++)
        ptr2->arr[i] = ptr1->arr[idx + i];

    ptr1->n = idx;
    int *temp = realloc(ptr1->arr, ptr1->n * sizeof(int));

    if (temp != NULL || ptr1->n == 0)
        ptr1->arr = temp;
}

void sort(Array *ptr) {
    if (ptr->arr == NULL || ptr->n == 0)
        return;

    for (int i = 1; i < ptr->n; i++) {
        int key = ptr->arr[i];
        int j = i - 1;

        while (j >= 0 && ptr->arr[j] > key) {
            ptr->arr[j + 1] = ptr->arr[j];
            j--;
        }
        ptr->arr[j + 1] = key;
    }
}

int main()
{
    srand(time(NULL));

    Array ptr[MAX];
    for(int i=0;i<MAX;i++){
        ptr[i].arr=NULL;
        ptr[i].n=0;
    }

    int current=-1;
    int total=0;

```

```

bool run=true;

while(run)
{
    printf("\n=====\\n");
    if(current== -1)
        printf("Current Array : None\\n");
    else
        printf("Current Array : A%d (Size=%d)\\n",current,ptr[current].n);
    printf("Total Arrays : %d/%d\\n",total,MAX);
    printf("=====\\n");

    printf("1. Create New Array\\n");
    printf("2. Switch Current Array\\n");
    printf("3. Display Current\\n");
    printf("4. Count\\n");
    printf("5. Reverse\\n");
    printf("6. Index Of\\n");
    printf("7. Get Element\\n");
    printf("8. Insert\\n");
    printf("9. Delete Element\\n");
    printf("10. Merge With Another Array\\n");
    printf("11. Split Current Array\\n");
    printf("12. Sort\\n");
    printf("13. Display All Arrays\\n");
    printf("14. Delete Current Array Entirely\\n");
    printf("15. Exit\\n");
    printf("Choice : ");

    int ch;
    if(scanf("%d",&ch) != 1) break;

    if(ch!=1 && ch!=2 && ch!=13 && ch!=15 && current== -1){
        printf("No active array. Create one first.\\n");
        continue;
    }

    switch(ch)
    {
    case 1:{
        if(total==MAX){ printf("Maximum arrays reached.\\n"); break; }
        int n;
        printf("Size : ");
        scanf("%d",&n);
        ptr[total]=create(n);
        current=total;
        total++;

        printf("Resulting Array A%d: ", current);
        display(ptr[current]);
        break;
    }
}

```

```

case 2:{
    if(total==0){ printf("No arrays.\n"); break; }
    printf("Existing Arrays\n");
    for(int i=0;i<total;i++)
        printf("A%d (Size=%d)\n",i,ptr[i].n);
    printf("Choose : ");
    scanf("%d",&current);
    if(current<0||current>=total){
        printf("Invalid.\n");
        current=0;
    }
    break;
}

case 3: display(ptr[current]); break;
case 4: printf("Count = %d\n",count(ptr[current])); break;
case 5:
    display(ptr[current]);
    reverse(&ptr[current]);
    display(ptr[current]);
    break;

case 6:{
    int t;
    printf("Target : ");
    scanf("%d",&t);
    Array pos=indexOf(ptr[current],t);
    if (pos.arr != NULL) {
        printf("Found at indexes: ");
        display(pos);
    }else if (pos.arr == NULL && ptr[current].arr != NULL) {
        printf("Item Not Found.\n");
    }
    free(pos.arr);
    break;
}

case 7:{
    int idx;
    printf("Index : ");
    scanf("%d",&idx);
    int x=getElement(ptr[current],idx);
    if(x==-1) printf("Array Empty\n");
    else if(x==-2) printf("Index Out of Bound\n");
    else printf("Element=%d\n",x);
    break;
}

case 8:{
    int v;
    printf("Value : ");
    scanf("%d",&v);
    ptr[current]=insert(ptr[current],v);
}

```



```

        display(ptr[current]);
        break;
    }

    case 9:{
        int v=-1;
        ptr[current]=delete(ptr[current],&v);
        if(v!=-1) printf("Deleted=%d\n",v);
        display(ptr[current]);
        break;
    }

    case 10:{
        int other;
        printf("Merge with Array : ");
        scanf("%d",&other);
        if(other<0||other>=total||other==current){ printf("Invalid.\n");
break; }

        ptr[current]=merge(&ptr[current],&ptr[other]);
        for(int i = other; i < total - 1; i++) {
            ptr[i] = ptr[i+1];
        }
        total--;
        if(current > other) {
            current--;
        }
        printf("Merged Array A%d: ", current);
        display(ptr[current]);
        break;
    }

    case 11:{
        if(total==MAX){ printf("Maximum arrays reached.\n"); break; }
        int idx;
        printf("Split Index : ");
        scanf("%d",&idx);

        if (idx < 0 || idx >= ptr[current].n) {
            printf("Invalid Index.\n");
            break;
        }

        split(&ptr[current],&ptr[total],idx);
        int split_index = total;
        total++;
        printf("Split Result -> A%d (Original Left): ", current);
        display(ptr[current]);
        printf("Split Result -> A%d (New Right): ", split_index);
        display(ptr[split_index]);

        current = split_index;
        break;
    }

```

```

    }

    case 12:
        sort(&ptr[current]);
        display(ptr[current]);
        break;

    case 13:
        for(int i=0;i<total;i++){
            printf("A%d : ",i);
            display(ptr[i]);
        }
        break;

    case 14:
        free(ptr[current].arr);
        for(int i = current; i < total - 1; i++) {
            ptr[i] = ptr[i+1];
        }
        ptr[total-1].arr = NULL;
        ptr[total-1].n = 0;

        total--;
        printf("Current array deleted completely.\n");

        if(total == 0) {
            current = -1;
        } else if (current >= total) {
            current = total - 1;
        }
        break;

    default:
        run=false;
        printf("Exiting Program...\n");
    }
}

for(int i=0;i<MAX;i++)
    free(ptr[i].arr);

return 0;
}

```

OUTPUT :

```
=====
Current Array : None
Total Arrays  : 0/20
=====
```

```
1. Create New Array
2. Switch Current Array
3. Display Current
4. Count
5. Reverse
6. Index Of
7. Get Element
8. Insert
9. Delete Element
10. Merge With Another Array
11. Split Current Array
12. Sort
13. Display All Arrays
14. Delete Current Array Entirely
15. Exit
Choice : 1
Size : 10
Generating random elements : 534 111 164 292 637 183 389 691 432 762
Array created successfully.
Resulting Array A0: [ 534 111 164 292 637 183 389 691 432 762 ]
```

```
=====
Current Array : A0 (Size=10)
Total Arrays  : 1/20
=====
```

```
1. Create New Array
2. Switch Current Array
3. Display Current
4. Count
5. Reverse
6. Index Of
7. Get Element
8. Insert
9. Delete Element
10. Merge With Another Array
11. Split Current Array
12. Sort
13. Display All Arrays
14. Delete Current Array Entirely
15. Exit
Choice : 5
[ 534 111 164 292 637 183 389 691 432 762 ]
Successfully reversed.
[ 762 432 691 389 183 637 292 164 111 534 ]
```

=====

Current Array : A0 (Size=10)

Total Arrays : 1/20

=====

1. Create New Array
2. Switch Current Array
3. Display Current
4. Count
5. Reverse
6. Index Of
7. Get Element
8. Insert
9. Delete Element
10. Merge With Another Array
11. Split Current Array
12. Sort
13. Display All Arrays
14. Delete Current Array Entirely
15. Exit

Choice : 8

Value : 100

Successfully Inserted.

[762 432 691 389 183 637 292 164 111 534 100]

=====

Current Array : A0 (Size=11)

Total Arrays : 1/20

=====

1. Create New Array
2. Switch Current Array
3. Display Current
4. Count
5. Reverse
6. Index Of
7. Get Element
8. Insert
9. Delete Element
10. Merge With Another Array
11. Split Current Array
12. Sort
13. Display All Arrays
14. Delete Current Array Entirely
15. Exit

Choice : 9

Deleted=100

[762 432 691 389 183 637 292 164 111 534]

=====

Current Array : A0 (Size=10)

Total Arrays : 1/20

=====

1. Create New Array
2. Switch Current Array
3. Display Current
4. Count
5. Reverse
6. Index Of
7. Get Element
8. Insert
9. Delete Element
10. Merge With Another Array
11. Split Current Array
12. Sort
13. Display All Arrays
14. Delete Current Array Entirely
15. Exit

Choice : 11

Split Index : 5

Split Result -> A0 (Original Left): [762 432 691 389 183]

Split Result -> A1 (New Right): [637 292 164 111 534]

=====

Current Array : A1 (Size=5)

Total Arrays : 2/20

=====

1. Create New Array
2. Switch Current Array
3. Display Current
4. Count
5. Reverse
6. Index Of
7. Get Element
8. Insert
9. Delete Element
10. Merge With Another Array
11. Split Current Array
12. Sort
13. Display All Arrays
14. Delete Current Array Entirely
15. Exit

Choice : 10

Merge with Array : 0

Merge Done.

Merged Array A0: [637 292 164 111 534 762 432 691 389 183]

Current Array : A1 (Size=14)

Total Arrays : 2/20

1. Create New Array
2. Switch Current Array
3. Display Current
4. Count
5. Reverse
6. Index Of
7. Get Element
8. Insert
9. Delete Element
10. Merge With Another Array
11. Split Current Array
12. Sort
13. Display All Arrays
14. Delete Current Array Entirely
15. Exit

Choice : 13

A0 : [111 164 183 292 389 432 534 637 691 762]

A1 : [604 150 328 582 530 887 632 262 171 926 417 625 959 784]

Current Array : None

Total Arrays : 0/20

1. Create New Array
2. Switch Current Array
3. Display Current
4. Count
5. Reverse
6. Index Of
7. Get Element
8. Insert
9. Delete Element
10. Merge With Another Array
11. Split Current Array
12. Sort
13. Display All Arrays
14. Delete Current Array Entirely
15. Exit

Choice : 15

Exiting Program...

Current Array : A0 (Size=10)

Total Arrays : 1/20

1. Create New Array
2. Switch Current Array
3. Display Current
4. Count
5. Reverse
6. Index Of
7. Get Element
8. Insert
9. Delete Element
10. Merge With Another Array
11. Split Current Array
12. Sort
13. Display All Arrays
14. Delete Current Array Entirely
15. Exit

Choice : 14

Current array deleted completely.

QUESTION 2: Write a menu-driven program that performs the following on a single linked list. (Write a function for each of the operations)

- I. Create a linked list.**
- II. Print the content of the list.**
- III. Insert an element at the front of the list**
- IV. Insert an element at the end of the list**
- V. Insert a node after the kth node.**
- VI. Insert a node after the node (first from the start) containing a given value.**
- VII. Insert a node before the kth node.**
- VIII. Insert a node before the node (first from the start) containing a given value.**
- IX. Delete the first node.**
- X. Delete the last node.**
- XI. Delete a node after the kth node.**
- XII. Delete a node before the kth node.**
- XIII. Delete the kth node.**
- XIV. Delete the node(first from the start) containing a specified value.**
- XV. Find the reverse of a list(not just printing in reverse)**
- XVI. Sort the list**
- XVII. Search a given element**

XVIII. Merge two lists; those are in ascending order.

XIX. Concatenate two list

XX. Find if two lists are equal(Boolean output)

SOURCE CODE :

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node{
    int val;
    struct Node *next;
}Node;

void freeList(Node* head) {
    Node* temp;
    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp);
    }
}

Node* create(int val) {
    Node* head = (Node*)malloc(sizeof(Node));

    if (head == NULL) {
        printf("Memory Allocation Failed.\n");
        return NULL;
    }

    head->next = NULL;
    head->val = val;
    return head;
}

void print(Node* head) {
    if (head == NULL) {
        printf("List is Empty.\n");
        return;
    }
    Node* temp = head;
    printf("[%p] -> ", temp);
    while (temp != NULL)
    {
        if (temp->next == NULL) {
            printf("[%d | %p]", temp->val, temp->next);
        } else {
            printf("[%d | %p] -> ", temp->val, temp->next);
        }
        temp = temp->next;
    }
    printf("\n");
}
```

```

Node* insertFront(Node* head, int val) {
    if (head == NULL) return create(val);

    Node* newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("Memory Allocation Failed.\n");
        return head;
    }
    newNode->val = val;
    newNode->next = head;
    head = newNode;
    return head;
}

Node* insertEnd(Node* head, int val) {
    if (head == NULL) return create(val);

    Node* newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("Memory Allocation Failed.\n");
        return head;
    }
    newNode->val = val;
    newNode->next = NULL;
    Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
    return head;
}

Node* insertAfter(Node* head, int val, int pos) {
    if (pos == 0 && head == NULL) return create(val);
    if (pos == 0 && head != NULL) return insertFront(head, val);

    Node* temp = head;
    int flag = 0;
    while (temp != NULL) {
        pos--;
        if (pos == 0) {
            flag = 1;
            break;
        }
        temp = temp->next;
    }
    if (!flag) {
        printf("Insertion Failed. Position out of range.\n");
        return head;
    }

    Node* newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("Memory Allocation Failed.\n");
    }

```



```

        return head;
    }
    newNode->val = val;
    newNode->next = temp->next;
    temp->next = newNode;
    return head;
}

Node* insertAfterValue(Node* head, int val, int pos) {
    Node* temp = head;
    int flag = 0;
    while (temp != NULL) {
        if (temp->val == pos) {
            flag = 1;
            break;
        }
        temp = temp->next;
    }
    if (!flag) {
        printf("Value does not matched.\n");
        return head;
    }
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("Memory Allocation Failed.\n");
        return head;
    }
    newNode->val = val;
    newNode->next = temp->next;
    temp->next = newNode;
    return head;
}

Node* insertBefore(Node* head, int val, int pos) {
    if (head == NULL && pos == 1) return create(val);
    if (head == NULL) {
        printf("Insertion Failed.\n");
        return head;
    }
    if (pos == 1) return insertFront(head, val);
    Node* temp = head;
    int flag = 0;
    pos -= 2;
    while (temp != NULL) {
        pos--;
        if (pos == 0) {
            flag = 1;
            break;
        }
        temp = temp->next;
    }
    if (!flag || temp == NULL) {
        printf("Insertion Failed. Position out of Bound.\n");
    }
}

```



```

        return head;
    }
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("Memory Allocation Failed.\n");
        return head;
    }
    newNode->val = val;
    newNode->next = temp->next;
    temp->next = newNode;
    return head;
}
Node* insertBeforeValue(Node* head, int val, int pos) {
    if (head == NULL) {
        printf("List is Empty.\n");
        return head;
    }
    if (head->val == pos) return insertFront(head, val);
    Node* temp = head;
    int flag = 0;
    while (temp->next != NULL) {
        if (temp->next->val == pos) {
            flag = 1;
            break;
        }
        temp = temp->next;
    }
    if (!flag) {
        printf("Value does not matched.\n");
        return head;
    }
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("Memory Allocation Failed.\n");
        return head;
    }
    newNode->val = val;
    newNode->next = temp->next;
    temp->next = newNode;
    return head;
}

Node* deleteFront(Node* head) {
    if (head == NULL) return head;
    Node* temp = head;
    head = head->next;
    free(temp);
    return head;
}

Node* deleteEnd(Node* head) {
    if (head == NULL) return head;
    if (head->next == NULL) {
        free(head);
    }
}

```

```

        return NULL;
    }
    Node* temp = head;
    while(temp->next->next != NULL) {
        temp = temp->next;
    }
    free(temp->next);
    temp->next = NULL;
    return head;
}

Node* deleteAfterKth(Node* head, int pos) {
    if (head == NULL) {
        printf("List is Empty.\n");
        return head;
    }
    Node* temp = head;
    while (temp != NULL && pos > 1) {
        temp = temp->next;
        pos--;
    }
    if (temp == NULL || temp->next == NULL) {
        printf("Deletion Failed. Position out of Bound.\n");
        return head;
    }
    Node* t = temp->next;
    temp->next = temp->next->next;
    free(t);
    return head;
}

Node* deleteBeforeKth(Node* head, int pos) {
    if (head == NULL || pos <= 1) {
        printf("Deletion Failed.\n");
        return head;
    }
    if (pos == 2) return deleteFront(head);
    Node* temp = head;
    while (temp != NULL && pos > 3) {
        temp = temp->next;
        pos--;
    }
    if (temp == NULL || temp->next == NULL) {
        printf("Deletion Failed. Position out of Bound.\n");
        return head;
    }
    Node* t = temp->next;
    temp->next = temp->next->next;
    free(t);
    return head;
}

Node* deleteKth(Node* head, int pos) {

```

```

    if (head == NULL) {
        printf("List is Empty.\n");
        return head;
    }
    if (pos == 1) return deleteFront(head);
    Node* temp = head;
    while (temp != NULL && pos > 2) {
        temp = temp->next;
        pos--;
    }
    if (temp == NULL || temp->next == NULL) {
        printf("Deletion Failed. Position out of Bound.\n");
        return head;
    }
    Node* t = temp->next;
    temp->next = temp->next->next;
    free(t);
    return head;
}

Node* deleteNodeValue(Node* head, int val) {
    if (head == NULL) {
        printf("List is Empty.\n");
        return head;
    }
    if (head->val == val) return deleteFront(head);
    Node* temp = head;
    int flag = 0;
    while (temp->next != NULL) {
        if (temp->next->val == val) {
            flag = 1;
            break;
        }
        temp = temp->next;
    }
    if (!flag) {
        printf("Deletion Failed. Value does not matched.\n");
        return head;
    }
    Node* t = temp->next;
    temp->next = temp->next->next;
    free(t);
    return head;
}

Node* reverse(Node* head) {
    Node* prev = NULL;
    Node* curr = head;
    Node* next = NULL;
    while (curr != NULL) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
}

```

```

    }
    return prev;
}
Node* sort(Node* head) {
    if (head == NULL) return head;
    Node* i;
    Node* j;
    for (i = head; i != NULL; i = i->next) {
        Node* min = i;
        for (j = i->next; j != NULL; j = j->next) {
            if (j->val < min->val) {
                min = j;
            }
        }
        if (min != i) {
            int temp = i->val;
            i->val = min->val;
            min->val = temp;
        }
    }
    return head;
}
int search(Node* head, int val) {
    Node* temp = head;
    int pos = -1;
    while (temp != NULL) {
        pos++;
        if (temp->val == val) {
            return pos;
        }
        temp = temp->next;
    }
    return -1;
}

```

```

Node* merge(Node* head1, Node* head2) {
    if (head1 == NULL) return sort(head2);
    if (head2 == NULL) return sort(head1);

    head1 = sort(head1);
    head2 = sort(head2);

    Node* temp1;
    Node* temp2 = head2;

    while (temp2 != NULL) {
        Node* nextNode = temp2->next;
        if (temp2->val < head1->val) {
            temp2->next = head1;
            head1 = temp2;
        } else {

```

```

        temp1 = head1;
        while (temp1->next != NULL && temp1->next->val < temp2->val) {
            temp1 = temp1->next;
        }

        temp2->next = temp1->next;
        temp1->next = temp2;
    }

    temp2 = nextNode;
}

return head1;
}

Node* concatenate(Node* head1, Node* head2) {
    if (head1 == NULL) return head2;
    if (head2 == NULL) return head1;
    Node* temp = head1;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = head2;
    return head1;
}

int isEqual(Node* head1, Node* head2) {
    int flag = 1;
    if (head1 == head2) return flag;
    while (head1 != NULL && head2 != NULL) {
        if (head1->val != head2->val) {
            flag = 0;
            break;
        }
        head1 = head1->next;
        head2 = head2->next;
    }

    if (flag == 1 && head1 == head2) {
        return flag;
    } else {
        return 0;
    }
}

Node* createAnother() {
    int size, val;
    Node* head = NULL;
    printf("Enter the size : ");
    scanf("%d",&size);
    for (int i = 1; i <= size; i++) {
        printf("Enter %dth element : ", i);
        scanf("%d",&val);
        head = insertEnd(head, val);
    }
}

```

```

    return head;
}

int main()
{
    int flag = 1;
    int choice, val, pos;
    Node *head = NULL, *head1 = NULL;

    do {
        printf("\n=====Press\n");
        printf("1. create\n");
        printf("2. print\n");
        printf("3. insert at front\n");
        printf("4. insert at end\n");
        printf("5. insert after kth node\n");
        printf("6. insert a node after value\n");
        printf("7. insert before kth node\n");
        printf("8. insert a node before value\n");
        printf("9. delete first node\n");
        printf("10. delete the last node\n");
        printf("11. delete a node after the kth node\n");
        printf("12. delete a node before kth node\n");
        printf("13. delete the kth node\n");
        printf("14. delete the node with value\n");
        printf("15. reverse\n");
        printf("16. sort\n");
        printf("17. search\n");
        printf("18. merge\n");
        printf("19. concatenate\n");
        printf("20. Equal\n");
        printf("21. exit\n");
        printf("\n=====>>> ");

        if (scanf("%d", &choice) != 1) {
            printf("\nError: Please enter a valid numerical option.\n\n");
            int c;
            while ((c = getchar()) != '\n' && c != EOF);
            continue;
        }

        switch (choice)
        {
            case 1:
                printf("Enter the value : ");
                scanf("%d",&val);
                head = create(val);
                print(head);
                break;

            case 2:
                print(head);

```

```

        break;

    case 3:
        printf("Enter the value : ");
        scanf("%d",&val);
        head = insertFront(head, val);
        print(head);
        break;

    case 4:
        printf("Enter the value : ");
        scanf("%d",&val);
        head = insertEnd(head, val);
        print(head);
        break;

    case 5:
        printf("Enter the position (after) : ");
        scanf("%d",&pos);
        printf("Enter the value : ");
        scanf("%d",&val);
        head = insertAfter(head, val, pos);
        print(head);
        break;

    case 6:
        print(head);
        printf("Enter the position value : ");
        scanf("%d",&pos);
        printf("Enter the value : ");
        scanf("%d",&val);
        head = insertAfterValue(head, val, pos);
        print(head);
        break;

    case 7:
        printf("Enter the position (before) : ");
        scanf("%d",&pos);
        printf("Enter the value : ");
        scanf("%d",&val);
        head = insertBefore(head, val, pos);
        print(head);
        break;

    case 8:
        print(head);
        printf("Enter the position value : ");
        scanf("%d",&pos);
        printf("Enter the value : ");
        scanf("%d",&val);
        head = insertBeforeValue(head, val, pos);
        print(head);
        break;

```

```

case 9:
    head = deleteFront(head);
    print(head);
    break;

case 10:
    head = deleteEnd(head);
    print(head);
    break;

case 11:
    printf("Enter kth position : ");
    scanf("%d",&pos);
    head = deleteAfterKth(head, pos);
    print(head);
    break;

case 12:
    printf("Enter kth position : ");
    scanf("%d",&pos);
    head = deleteBeforeKth(head, pos);
    print(head);
    break;

case 13:
    printf("Enter kth position : ");
    scanf("%d",&pos);
    head = deleteKth(head, pos);
    print(head);
    break;

case 14:
    printf("Enter value : ");
    scanf("%d",&val);
    head = deleteNodeValue(head, val);
    print(head);
    break;

case 15:
    head = reverse(head);
    print(head);
    break;

case 16:
    head = sort(head);
    print(head);
    break;

case 17:
    printf("Enter the val : ");
    scanf("%d",&val);
    pos = search(head, val);

```



```

        if (pos != -1) {
            printf("Item found at %d index.",pos);
        } else {
            printf("Item not found.");
        }
        break;

    case 18:
        head1 = createAnother();
        head = merge(head, head1);
        print(head);
        break;

    case 19:
        head1 = createAnother();
        printf("Before concatenation : \n");
        print(head);
        print(head1);
        head = concatenate(head, head1);
        printf("After concatenation : \n");
        print(head);
        break;

    case 20:
        head1 = createAnother();
        print(head);
        print(head1);
        if (isEqual(head, head1)) {
            printf("Equal.");
        } else {
            printf("Unequal.");
        }
        break;

    case 21:
        flag = 0;
        break;

    default:
        printf("Wrong Input.\n");
        break;
}

} while (flag == 1);

freeList(head1);
freeList(head);
return 0;
}

```

OUTPUT :

=====

Press

1. create
2. print
3. insert at front
4. insert at end
5. insert after kth node
6. insert a node after value
7. insert before kth node
8. insert a node before value
9. delete first node
10. delete the last node
11. delete a node after the kth node
12. delete a node before kth node
13. delete the kth node
14. delete the node with value
15. reverse
16. sort
17. search
18. merge
19. concatenate
20. Equal
21. exit

=====

>>> 1

Enter the value : 10

[009818B0] -> [10 | 00000000]

=====

Press

1. create
2. print
3. insert at front
4. insert at end
5. insert after kth node
6. insert a node after value
7. insert before kth node
8. insert a node before value
9. delete first node
10. delete the last node
11. delete a node after the kth node
12. delete a node before kth node
13. delete the kth node
14. delete the node with value
15. reverse
16. sort
17. search
18. merge
19. concatenate
20. Equal
21. exit

=====

>>> 4

Enter the value : 30

[009818C0] -> [20 | 009818B0] -> [10 | 009818D0] -> [30 | 00000000]

=====

Press

1. create
2. print
3. insert at front
4. insert at end
5. insert after kth node
6. insert a node after value
7. insert before kth node
8. insert a node before value
9. delete first node
10. delete the last node
11. delete a node after the kth node
12. delete a node before kth node
13. delete the kth node
14. delete the node with value
15. reverse
16. sort
17. search
18. merge
19. concatenate
20. Equal
21. exit

=====

>>> 3

Enter the value : 20

[009818C0] -> [20 | 009818B0] -> [10 | 00000000]

=====

Press

1. create
2. print
3. insert at front
4. insert at end
5. insert after kth node
6. insert a node after value
7. insert before kth node
8. insert a node before value
9. delete first node
10. delete the last node
11. delete a node after the kth node
12. delete a node before kth node
13. delete the kth node
14. delete the node with value
15. reverse
16. sort
17. search
18. merge
19. concatenate
20. Equal
21. exit

=====

>>> 6

[009818C0] -> [20 | 009818B0] -> [10 | 009818D0] -> [30 | 00000000]

Enter the position value : 20

Enter the value : 40

[009818C0] -> [20 | 009817E8] -> [40 | 009818B0] -> [10 | 009818D0] -> [30 | 00000000]

=====

Press

1. create
2. print
3. insert at front
4. insert at end
5. insert after kth node
6. insert a node after value
7. insert before kth node
8. insert a node before value
9. delete first node
10. delete the last node
11. delete a node after the kth node
12. delete a node before kth node
13. delete the kth node
14. delete the node with value
15. reverse
16. sort
17. search
18. merge
19. concatenate
20. Equal
21. exit

=====

>>> 9

[009817E8] -> [40 | 00981808] -> [60 | 009817F8] -> [50 | 009818B0] -> [10 | 009818D0] -> [30 | 00000000]

=====

Press

1. create
2. print
3. insert at front
4. insert at end
5. insert after kth node
6. insert a node after value
7. insert before kth node
8. insert a node before value
9. delete first node
10. delete the last node
11. delete a node after the kth node
12. delete a node before kth node
13. delete the kth node
14. delete the node with value
15. reverse
16. sort
17. search
18. merge
19. concatenate
20. Equal
21. exit

=====

>>> 10

[009817E8] -> [40 | 00981808] -> [60 | 009817F8] -> [50 | 009818B0] -> [10 | 00000000]

=====

Press

1. create
2. print
3. insert at front
4. insert at end
5. insert after kth node
6. insert a node after value
7. insert before kth node
8. insert a node before value
9. delete first node
10. delete the last node
11. delete a node after the kth node
12. delete a node before kth node
13. delete the kth node
14. delete the node with value
15. reverse
16. sort
17. search
18. merge
19. concatenate
20. Equal
21. exit

=====

>>> 15

[009818B0] -> [10 | 009817F8] -> [50 | 00981808] -> [60 | 009817E8] -> [40 | 00000000]

=====

Press

1. create
2. print
3. insert at front
4. insert at end
5. insert after kth node
6. insert a node after value
7. insert before kth node
8. insert a node before value
9. delete first node
10. delete the last node
11. delete a node after the kth node
12. delete a node before kth node
13. delete the kth node
14. delete the node with value
15. reverse
16. sort
17. search
18. merge
19. concatenate
20. Equal
21. exit

=====

>>> 16

[009818B0] -> [10 | 009817F8] -> [40 | 00981808] -> [50 | 009817E8] -> [60 | 00000000]


```
=====
Press
1. create
2. print
3. insert at front
4. insert at end
5. insert after kth node
6. insert a node after value
7. insert before kth node
8. insert a node before value
9. delete first node
10. delete the last node
11. delete a node after the kth node
12. delete a node before kth node
13. delete the kth node
14. delete the node with value
15. reverse
16. sort
17. search
18. merge
19. concatenate
20. Equal
21. exit
```

```
=====
>>> 17
```

```
Enter the val : 50
```

```
Item found at 2 index.
```

```
=====
Press
1. create
2. print
3. insert at front
4. insert at end
5. insert after kth node
6. insert a node after value
7. insert before kth node
8. insert a node before value
9. delete first node
10. delete the last node
11. delete a node after the kth node
12. delete a node before kth node
13. delete the kth node
14. delete the node with value
15. reverse
16. sort
17. search
18. merge
19. concatenate
20. Equal
21. exit
```

```
=====
>>> 21
```

QUESTION 3 : Write a menu-driven program representing a polynomial as a data structure using a singly linked list and write functions to add, subtract and multiply two polynomials.

SOURCE CODE :

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int cof;
    int pwr;
    char var;
    struct Node *next;
} Node;

Node* insertTerm(Node* head, int cof, int pwr, char var) {
    if (cof == 0) return head;

    Node *newNode, *temp, *prev;

    temp = head;
    while (temp != NULL) {
        if (temp->pwr == pwr) {
            temp->cof += cof;

            if (temp->cof == 0) {
                if (temp == head) {
                    head = head->next;
                    free(temp);
                } else {
                    prev->next = temp->next;
                    free(temp);
                }
            }
            return head;
        }
        prev = temp;
        temp = temp->next;
    }
    newNode = (Node*)malloc(sizeof(Node));
    newNode->cof = cof;
    newNode->pwr = pwr;
    newNode->var = var;
    newNode->next = NULL;

    if (head == NULL || pwr > head->pwr) {
        newNode->next = head;
        return newNode;
    }
    temp = head;

    while (temp->next != NULL && temp->next->pwr > pwr) {
        temp = temp->next;
    }
```

```

    }

    newNode->next = temp->next;
    temp->next = newNode;

    return head;
}

Node* createPoly() {
    int n;
    printf("Enter number of terms: ");
    scanf("%d", &n);

    Node* head = NULL;

    for (int i = 0; i < n; i++) {
        int cof, pwr;
        printf("\nTerm %d\n", i + 1);

        printf("Coefficient: ");
        scanf("%d", &cof);

        printf("Power: ");
        scanf("%d", &pwr);
        head = insertTerm(head, cof, pwr, 'x');
    }

    return head;
}

void display(Node* head) {
    if (head == NULL) {
        printf("0\n");
        return;
    }
    Node* temp = head;
    int first = 1;

    while (temp != NULL) {
        if (!first && temp->cof > 0) printf(" + ");
        if (temp->cof < 0) printf(" - ");
        int coef = temp->cof < 0 ? -temp->cof : temp->cof;
        if (temp->next == NULL) {
            printf("%d", coef);
        } else {
            printf("%d%c^%d", coef, temp->var, temp->pwr);
        }

        first = 0;
        temp = temp->next;
    }
    printf("\n");
}

Node* add(Node* p1, Node* p2) {

```



```

Node* res = NULL;
while (p1 != NULL) {
    res = insertTerm(res, p1->cof, p1->pwr, p1->var);
    p1 = p1->next;
}
while (p2 != NULL) {
    res = insertTerm(res, p2->cof, p2->pwr, p2->var);
    p2 = p2->next;
}
return res;
}

Node* sub(Node* p1, Node* p2) {
    Node* res = NULL;
    while (p1 != NULL) {
        res = insertTerm(res, p1->cof, p1->pwr, p1->var);
        p1 = p1->next;
    }
    while (p2 != NULL) {
        res = insertTerm(res, -p2->cof, p2->pwr, p2->var);
        p2 = p2->next;
    }
    return res;
}

Node* mul(Node* p1, Node* p2) {
    Node* res = NULL;
    Node* t1 = p1;

    while (t1 != NULL) {
        Node* t2 = p2;
        while (t2 != NULL) {
            int coef = t1->cof * t2->cof;
            int power = t1->pwr + t2->pwr;
            res = insertTerm(res, coef, power, 'x');
            t2 = t2->next;
        }
        t1 = t1->next;
    }

    return res;
}

void freeList(Node* head) {
    Node* temp;

    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp);
    }
}

int main() {

```

```

Node *p1 = NULL, *p2 = NULL, *result = NULL;

int choice;

do {
    printf("\n===== POLYNOMIAL MENU =====\n");
    printf("1. Create Polynomial 1\n");
    printf("2. Create Polynomial 2\n");
    printf("3. Display Polynomials\n");
    printf("4. Add\n");
    printf("5. Subtract (P1 - P2)\n");
    printf("6. Multiply\n");
    printf("0. Exit\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    switch (choice) {

        case 1:
            freeList(p1);
            p1 = createPoly();
            break;

        case 2:
            freeList(p2);
            p2 = createPoly();
            break;

        case 3:
            printf("\nP1 = ");
            display(p1);

            printf("P2 = ");
            display(p2);
            break;

        case 4:
            freeList(result);
            result = add(p1, p2);

            printf("\nP1 + P2 = ");
            display(result);
            break;

        case 5:
            freeList(result);
            result = sub(p1, p2);

            printf("\nP1 - P2 = ");
            display(result);
            break;
    }
}

```

```

        case 6:
            freeList(result);
            result = mul(p1, p2);

            printf("\nP1 * P2 = ");
            display(result);
            break;

        case 0:
            printf("Exiting...\n");
            break;

        default:
            printf("Invalid choice!\n");
    }

    } while (choice != 0);

    freeList(p1);
    freeList(p2);
    freeList(result);

    return 0;
}

```

OUTPUT :

```

===== POLYNOMIAL MENU =====
1. Create Polynomial 1
2. Create Polynomial 2
3. Display Polynomials
4. Add
5. Subtract (P1 - P2)
6. Multiply
0. Exit
Enter choice: 1
Enter number of terms: 3

Term 1
Coefficient: 3
Power: 3

Term 2
Coefficient: -5
Power: 2

Term 3
Coefficient: 10
Power: 0

```

```

===== POLYNOMIAL MENU =====
1. Create Polynomial 1
2. Create Polynomial 2
3. Display Polynomials
4. Add
5. Subtract (P1 - P2)
6. Multiply
0. Exit
Enter choice: 2
Enter number of terms: 4

Term 1
Coefficient: 2
Power: 4

Term 2
Coefficient: -4
Power: 3

Term 3
Coefficient: -7
Power: 1

Term 4
Coefficient: 10
Power: 0

```

```

===== POLYNOMIAL MENU =====
1. Create Polynomial 1
2. Create Polynomial 2
3. Display Polynomials
4. Add
5. Subtract (P1 - P2)
6. Multiply
0. Exit
Enter choice: 3

```

```

P1 = 3x^3 - 5x^2 + 10
P2 = 2x^4 - 4x^3 - 7x^1 + 10

```

```

===== POLYNOMIAL MENU =====
1. Create Polynomial 1
2. Create Polynomial 2
3. Display Polynomials
4. Add
5. Subtract (P1 - P2)
6. Multiply
0. Exit
Enter choice: 4

```

```

P1 + P2 = 2x^4 - 1x^3 - 5x^2 - 7x^1 + 20

```

===== POLYNOMIAL MENU =====

1. Create Polynomial 1
2. Create Polynomial 2
3. Display Polynomials
4. Add
5. Subtract (P1 - P2)
6. Multiply
0. Exit

Enter choice: 5

$$P1 - P2 = -2x^4 + 7x^3 - 5x^2 + 7$$

===== POLYNOMIAL MENU =====

1. Create Polynomial 1
2. Create Polynomial 2
3. Display Polynomials
4. Add
5. Subtract (P1 - P2)
6. Multiply
0. Exit

Enter choice: 6

$$P1 * P2 = 6x^7 - 22x^6 + 20x^5 - 1x^4 + 25x^3 - 50x^2 - 70x^1 + 100$$

===== POLYNOMIAL MENU =====

1. Create Polynomial 1
2. Create Polynomial 2
3. Display Polynomials
4. Add
5. Subtract (P1 - P2)
6. Multiply
0. Exit

Enter choice: 0

Exiting...

QUESTION 4 : Implement Doubly Linked List for the operations (I to XV)

- I. Create a linked list.
- II. Print the content of the list.
- III. Insert an element at the front of the list
- IV. Insert an element at the end of the list
- V. Insert a node after the kth node.
- VI. Insert a node after the node (first from the start) containing a given value.
- VII. Insert a node before the kth node.
- VIII. Insert a node before the node (first from the start) containing a given value.
- IX. Delete the first node.
- X. Delete the last node.
- XI. Delete a node after the kth node.
- XII. Delete a node before the kth node.
- XIII. Delete the kth node.
- XIV. Delete the node(first from the start) containing a specified value.
- XV. Find the reverse of a list(not just printing in reverse)

SOURCE CODE :

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
} Node;

Node* createNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = NULL;
    return newNode;
}

Node* createList() {
    int n, data;
    Node *head = NULL, *tail = NULL;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for(int i = 0; i < n; i++) {
        printf("Enter data: ");
        scanf("%d", &data);

        Node* newNode = createNode(data);

        if(head == NULL) {
            head = tail = newNode;
        }
        else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
    }
}
```

```

    }
}

return head;
}

void display(Node* head) {
    if(head == NULL) {
        printf("List is Empty\n");
        return;
    }

    printf("List: ");

    while(head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }

    printf("\n");
}

Node* insertFront(Node* head, int data) {
    Node* newNode = createNode(data);

    newNode->next = head;

    if(head != NULL)
        head->prev = newNode;

    return newNode;
}

Node* insertEnd(Node* head, int data) {
    Node* newNode = createNode(data);

    if(head == NULL)
        return newNode;

    Node* temp = head;

    while(temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
    newNode->prev = temp;

    return head;
}

Node* insertAfterK(Node* head, int k, int data) {
    Node* temp = head;

    for(int i = 1; i < k && temp != NULL; i++)

```

```

        temp = temp->next;

    if(temp == NULL) {
        printf("Invalid Position\n");
        return head;
    }

    Node* newNode = createNode(data);

    newNode->next = temp->next;
    newNode->prev = temp;

    if(temp->next != NULL)
        temp->next->prev = newNode;

    temp->next = newNode;

    return head;
}

Node* insertAfterValue(Node* head, int value, int data) {
    Node* temp = head;

    while(temp != NULL && temp->data != value)
        temp = temp->next;

    if(temp == NULL) {
        printf("Value not found\n");
        return head;
    }

    Node* newNode = createNode(data);

    newNode->next = temp->next;
    newNode->prev = temp;

    if(temp->next != NULL)
        temp->next->prev = newNode;

    temp->next = newNode;

    return head;
}

Node* insertBeforeK(Node* head, int k, int data) {
    if(k == 1)
        return insertFront(head, data);

    Node* temp = head;

    for(int i = 1; i < k && temp != NULL; i++)
        temp = temp->next;

```

```

    if(temp == NULL) {
        printf("Invalid Position\n");
        return head;
    }

    Node* newNode = createNode(data);

    newNode->next = temp;
    newNode->prev = temp->prev;

    temp->prev->next = newNode;
    temp->prev = newNode;

    return head;
}
Node* insertBeforeValue(Node* head, int value, int data) {
    Node* temp = head;

    while(temp != NULL && temp->data != value)
        temp = temp->next;

    if(temp == NULL) {
        printf("Value not found\n");
        return head;
    }

    if(temp == head)
        return insertFront(head, data);

    Node* newNode = createNode(data);

    newNode->next = temp;
    newNode->prev = temp->prev;

    temp->prev->next = newNode;
    temp->prev = newNode;

    return head;
}
Node* deleteFirst(Node* head) {
    if(head == NULL)
        return NULL;

    Node* temp = head;

    head = head->next;

    if(head != NULL)
        head->prev = NULL;

    free(temp);
}

```



```

    return head;
}

Node* deleteLast(Node* head) {
    if(head == NULL)
        return NULL;

    if(head->next == NULL) {
        free(head);
        return NULL;
    }

    Node* temp = head;

    while(temp->next != NULL)
        temp = temp->next;

    temp->prev->next = NULL;

    free(temp);

    return head;
}

Node* deleteAfterK(Node* head, int k) {
    Node* temp = head;

    for(int i = 1; i < k && temp != NULL; i++)
        temp = temp->next;

    if(temp == NULL || temp->next == NULL) {
        printf("Deletion not possible\n");
        return head;
    }

    Node* del = temp->next;

    temp->next = del->next;

    if(del->next != NULL)
        del->next->prev = temp;

    free(del);

    return head;
}

Node* deleteBeforeK(Node* head, int k) {
    if(k <= 2) {
        printf("Deletion not possible\n");
        return head;
    }
}

```

```

Node* temp = head;

for(int i = 1; i < k && temp != NULL; i++)
    temp = temp->next;

if(temp == NULL || temp->prev == NULL) {
    printf("Deletion not possible\n");
    return head;
}

Node* del = temp->prev;

if(del == head) {
    head = head->next;
    head->prev = NULL;
    free(del);
    return head;
}

del->prev->next = temp;
temp->prev = del->prev;

free(del);

return head;
}

Node* deleteK(Node* head, int k) {
    if(k == 1)
        return deleteFirst(head);

    Node* temp = head;

    for(int i = 1; i < k && temp != NULL; i++)
        temp = temp->next;

    if(temp == NULL) {
        printf("Invalid Position\n");
        return head;
    }

    if(temp->next != NULL)
        temp->next->prev = temp->prev;
    temp->prev->next = temp->next;
    free(temp);
    return head;
}

Node* deleteValue(Node* head, int value) {
    Node* temp = head;
    while(temp != NULL && temp->data != value)
        temp = temp->next;
    if(temp == NULL) {
        printf("Value not found\n");

```

```

        return head;
    }
    if(temp == head)
        return deleteFirst(head);
    if(temp->next != NULL)
        temp->next->prev = temp->prev;
    temp->prev->next = temp->next;
    free(temp);
    return head;
}

Node* reverseList(Node* head) {
    Node *temp = NULL;
    Node *current = head;
    while(current != NULL) {
        temp = current->prev;
        current->prev = current->next;
        current->next = temp;
        current = current->prev;
    }
    if(temp != NULL)
        head = temp->prev;
    return head;
}

void freeList(Node* head) {
    Node* temp;

    while(head != NULL) {
        temp = head;
        head = head->next;
        free(temp);
    }
}

int main() {
    Node* head = NULL;
    int choice, data, k, value;
    do {
        printf("\n===== DOUBLY LINKED LIST MENU =====\n");
        printf("1. Create List\n");
        printf("2. Display List\n");
        printf("3. Insert Front\n");
        printf("4. Insert End\n");
        printf("5. Insert After Kth Node\n");
        printf("6. Insert After Value\n");
        printf("7. Insert Before Kth Node\n");
        printf("8. Insert Before Value\n");
        printf("9. Delete First Node\n");
        printf("10. Delete Last Node\n");
        printf("11. Delete After Kth Node\n");
        printf("12. Delete Before Kth Node\n");
        printf("13. Delete Kth Node\n");
        printf("14. Delete Value\n");
        printf("15. Reverse List\n");
    }
}

```

```

printf("0. Exit\n");
printf("Enter Choice: ");
scanf("%d", &choice);
switch(choice) {
    case 1:
        freeList(head);
        head = createList();
        display(head);
        break;

    case 2:
        display(head);
        break;

    case 3:
        printf("Enter data: ");
        scanf("%d", &data);
        head = insertFront(head, data);
        display(head);
        break;

    case 4:
        printf("Enter data: ");
        scanf("%d", &data);
        head = insertEnd(head, data);
        display(head);
        break;

    case 5:
        printf("Enter k and data: ");
        scanf("%d%d", &k, &data);
        head = insertAfterK(head, k, data);
        display(head);
        break;

    case 6:
        printf("Enter value and data: ");
        scanf("%d%d", &value, &data);
        head = insertAfterValue(head, value, data);
        display(head);
        break;

    case 7:
        printf("Enter k and data: ");
        scanf("%d%d", &k, &data);
        head = insertBeforeK(head, k, data);
        display(head);
        break;

    case 8:
        printf("Enter value and data: ");
        scanf("%d%d", &value, &data);
        head = insertBeforeValue(head, value, data);

```

```

        display(head);
        break;

    case 9:
        head = deleteFirst(head);
        display(head);
        break;

    case 10:
        head = deleteLast(head);
        display(head);
        break;

    case 11:
        printf("Enter k: ");
        scanf("%d", &k);
        head = deleteAfterK(head, k);
        display(head);
        break;

    case 12:
        printf("Enter k: ");
        scanf("%d", &k);
        head = deleteBeforeK(head, k);
        display(head);
        break;

    case 13:
        printf("Enter k: ");
        scanf("%d", &k);
        head = deleteK(head, k);
        display(head);
        break;

    case 14:
        printf("Enter value: ");
        scanf("%d", &value);
        head = deleteValue(head, value);
        display(head);
        break;

    case 15:
        display(head);
        head = reverseList(head);
        printf("List Reversed Successfully\n");
        display(head);
        break;
    case 0:
        printf("Program Terminated.\n");
        break;
    default:
        printf("Invalid Choice!\n");
}

```

```
    } while(choice != 0);  
    freeList(head);  
    return 0;  
}
```

OUTPUT :

===== DOUBLY LINKED LIST MENU =====

```
1. Create List
2. Display List
3. Insert Front
4. Insert End
5. Insert After Kth Node
6. Insert After Value
7. Insert Before Kth Node
8. Insert Before Value
9. Delete First Node
10. Delete Last Node
11. Delete After Kth Node
12. Delete Before Kth Node
13. Delete Kth Node
14. Delete Value
15. Reverse List
0. Exit
Enter Choice: 1
Enter number of nodes: 3
Enter data: 10
Enter data: 20
Enter data: 30
List: 10 20 30
```

===== DOUBLY LINKED LIST MENU =====

```
1. Create List
2. Display List
3. Insert Front
4. Insert End
5. Insert After Kth Node
6. Insert After Value
7. Insert Before Kth Node
8. Insert Before Value
9. Delete First Node
10. Delete Last Node
11. Delete After Kth Node
12. Delete Before Kth Node
13. Delete Kth Node
14. Delete Value
15. Reverse List
0. Exit
Enter Choice: 3
Enter data: 40
List: 40 10 20 30
```

===== DOUBLY LINKED LIST MENU =====

```
1. Create List
2. Display List
3. Insert Front
4. Insert End
5. Insert After Kth Node
6. Insert After Value
7. Insert Before Kth Node
8. Insert Before Value
9. Delete First Node
10. Delete Last Node
11. Delete After Kth Node
12. Delete Before Kth Node
13. Delete Kth Node
14. Delete Value
15. Reverse List
0. Exit
Enter Choice: 3
Enter data: 50
List: 50 40 10 20 30
```

===== DOUBLY LINKED LIST MENU =====

```
1. Create List
2. Display List
3. Insert Front
4. Insert End
5. Insert After Kth Node
6. Insert After Value
7. Insert Before Kth Node
8. Insert Before Value
9. Delete First Node
10. Delete Last Node
11. Delete After Kth Node
12. Delete Before Kth Node
13. Delete Kth Node
14. Delete Value
15. Reverse List
0. Exit
Enter Choice: 5
Enter k and data: 2 60
List: 50 40 60 10 20 30
```

===== DOUBLY LINKED LIST MENU =====

```
1. Create List
2. Display List
3. Insert Front
4. Insert End
5. Insert After Kth Node
6. Insert After Value
7. Insert Before Kth Node
8. Insert Before Value
9. Delete First Node
10. Delete Last Node
11. Delete After Kth Node
12. Delete Before Kth Node
13. Delete Kth Node
14. Delete Value
15. Reverse List
0. Exit
Enter Choice: 6
Enter value and data: 10 70
List: 50 40 60 10 70 20 30
```

===== DOUBLY LINKED LIST MENU =====

```
1. Create List
2. Display List
3. Insert Front
4. Insert End
5. Insert After Kth Node
6. Insert After Value
7. Insert Before Kth Node
8. Insert Before Value
9. Delete First Node
10. Delete Last Node
11. Delete After Kth Node
12. Delete Before Kth Node
13. Delete Kth Node
14. Delete Value
15. Reverse List
0. Exit
Enter Choice: 9
List: 40 60 10 70 20 30
```

===== DOUBLY LINKED LIST MENU =====

```
1. Create List
2. Display List
3. Insert Front
4. Insert End
5. Insert After Kth Node
6. Insert After Value
7. Insert Before Kth Node
8. Insert Before Value
9. Delete First Node
10. Delete Last Node
11. Delete After Kth Node
12. Delete Before Kth Node
13. Delete Kth Node
14. Delete Value
15. Reverse List
0. Exit
Enter Choice: 10
List: 40 60 10 70 20
```

===== DOUBLY LINKED LIST MENU =====

```
1. Create List
2. Display List
3. Insert Front
4. Insert End
5. Insert After Kth Node
6. Insert After Value
7. Insert Before Kth Node
8. Insert Before Value
9. Delete First Node
10. Delete Last Node
11. Delete After Kth Node
12. Delete Before Kth Node
13. Delete Kth Node
14. Delete Value
15. Reverse List
0. Exit
Enter Choice: 15
List: 40 60 10 70 20
List Reversed Successfully
List: 20 70 10 60 40
```

===== DOUBLY LINKED LIST MENU =====

```
1. Create List
2. Display List
3. Insert Front
4. Insert End
5. Insert After Kth Node
6. Insert After Value
7. Insert Before Kth Node
8. Insert Before Value
9. Delete First Node
10. Delete Last Node
11. Delete After Kth Node
12. Delete Before Kth Node
13. Delete Kth Node
14. Delete Value
15. Reverse List
0. Exit
Enter Choice: 0
Program Terminated.
```

QUESTION 5: Given a problem, “people are standing in a circle waiting to be executed. Counting begins at a specified point in the circle and proceeds in a specified direction (e.g., clockwise). After a specified number of people is skipped, the next person is executed (i.e., removed). The procedure is repeated with the remaining people, starting with the next person, going in the same direction and skipping the same number of people until only one person remains and is freed”. The solution is to find the person to be freed. Solve it using

i) 2D array

SOURCE CODE :

```
#include <stdio.h>
int josephusArray(int n, int k) {
    int arr[n][2];
    for(int i=0;i<n;i++) {
        arr[i][0] = i+1;
        arr[i][1] = 1;
    }
    int alive = n;
    int pos = 0;
    int count = 0;

    while(alive > 1) {
        if(arr[pos][1] == 1) {
            count++;
            if(count == k) {
                arr[pos][1] = 0;
                printf("Executed: %d\n", arr[pos][0]);
                alive--;
                count = 0;
            }
        }
        pos = (pos + 1) % n;
    }
    for(int i=0;i<n;i++) {
        if(arr[i][1] == 1)
            return arr[i][0];
    }
    return -1;
}

int main() {
    int n,k;
    printf("Enter number of people: ");
    scanf("%d",&n);
    printf("Enter k: ");
    scanf("%d",&k);
    int survivor = josephusArray(n,k);
    printf("Freed Person = %d\n",survivor);
    return 0;
}
```

OUTPUT :


```

Enter number of people: 10
Enter k: 3
Executed: 3
Executed: 6
Executed: 9
Executed: 2
Executed: 7
Executed: 1
Executed: 8
Executed: 5
Executed: 10
Freed Person = 4

```

ii) A circular linked list.

SOURCE CODE :

```

#include<stdio.h>
#include<stdlib.h>
typedef struct Node{
    int data;
    struct Node* next;
}Node;
Node* createCircle(int n){
    Node *head=NULL,*temp,*newNode;
    for(int i=1;i<=n;i++) {
        newNode=(Node*)malloc(sizeof(Node));
        newNode->data=i;
        newNode->next=NULL;
        if(head==NULL) {
            head=newNode;
            temp=newNode;
        } else {
            temp->next=newNode;
            temp=newNode;
        }
    }
    temp->next=head;
    return head;
}
int josephusCLL(int n,int k){
    Node* head=createCircle(n);
    Node *ptr=head;
    Node *prev=NULL;
    while(ptr->next!=ptr) {
        for(int i=1;i<k;i++) {
            prev=ptr;
            ptr=ptr->next;
        }
        printf("Executed: %d\n",ptr->data);
        prev->next=ptr->next;
        Node* del=ptr;
        ptr=ptr->next;
        free(del);
    }
    int survivor=ptr->data;
    free(ptr);
    return survivor;
}
int main() {
    int n,k;
    printf("Enter n: ");

```

```

scanf("%d",&n);
printf("Enter k: ");
scanf("%d",&k);
printf("Freed Person = %d\n",josephusCLL(n,k));
return 0;
}

```

OUTPUT :

```

Enter n: 10
Enter k: 3
Executed: 3
Executed: 6
Executed: 9
Executed: 2
Executed: 7
Executed: 1
Executed: 8
Executed: 5
Executed: 10
Freed Person = 4

```

iii) A circular doubly linked list.

SOURCE CODE :

```

#include<stdio.h>
#include<stdlib.h>
typedef struct Node{
    int data;
    struct Node *prev;
    struct Node *next;
}Node;
Node* createCDLL(int n){
    Node *head=NULL,*last=NULL,*newNode;
    for(int i=1;i<=n;i++) {
        newNode=(Node*)malloc(sizeof(Node));
        newNode->data=i;
        if(head==NULL) {
            head=newNode;
            last=newNode;
        } else {
            last->next=newNode;
            newNode->prev=last;
            last=newNode;
        }
    }
    head->prev=last;
    last->next=head;
    return head;
}
int josephusCDLL(int n,int k){
    Node* ptr=createCDLL(n);
    int count=n;
    while(count>1){
        for(int i=1;i<k;i++)
            ptr=ptr->next;
        printf("Executed: %d\n",ptr->data);
        ptr->prev->next=ptr->next;
    }
}

```

```

        ptr->next->prev=ptr->prev;
        Node* del=ptr;
        ptr=ptr->next;
        free(del);
        count--;
    }
    int survivor=ptr->data;
    free(ptr);
    return survivor;
}
int main(){
    int n,k;
    printf("Enter n: ");
    scanf("%d",&n);
    printf("Enter k: ");
    scanf("%d",&k);
    printf("Freed Person = %d\n",josephusCDLL(n,k));
    return 0;
}

```

OUTPUT :

```

Enter n: 10
Enter k: 3
Executed: 3
Executed: 6
Executed: 9
Executed: 2
Executed: 7
Executed: 1
Executed: 8
Executed: 5
Executed: 10
Freed Person = 4

```

QUESTION 6: Write a program to implement a stack data structure using an array.

SOURCE CODE :

```
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>
#define MAX 10
int stack[MAX];
int top = -1;
bool isEmpty() {
    return top == -1;
}
bool isFull() {
    return top == MAX - 1;
}
void push(int val) {
    if (isFull()) {
        printf("Overflow.\n");
        return;
    }
    top++;
    stack[top] = val;
    printf("Insertion successfull.\n");
}
int pop() {
    if (isEmpty()) {
        printf("Underflow.\n");
        return 0;
    }
    int val = stack[top];
    stack[top] = 0;
    top--;
    return val;
}
int peek() {
    if (isEmpty()) {
        printf("Underflow.\n");
        return 0;
    }
    return stack[top];
}
void print() {
    printf("STACK TOP : ");
    for (int i = top; i >= 0; i--) {
        printf(" %d ", stack[i]);
    }
    printf("\n");
}
int main() {
    int ch, val;
    bool f = true;
    do {
        printf("Press\n1. push\n2. pop\n3. peek\n4. print\n5. exit\n>>> ");
        scanf("%d",&ch);
```

```

switch (ch) {
case 1:
    printf("Enter the value : ");
    scanf("%d",&val);
    push(val);
    print();
    break;
case 2:
    val = pop();
    if (val != 0) printf("Pop : %d\n", val);
    print();
    break;
case 3:
    val = peek();
    if (val != 0) printf("Peek : %d\n", val);
    print();
    break;
case 4:
    print();
    break;
case 5:
    f = false;
    break;
default:
    printf("Wrong Input.\n");
    break;
}
} while (f);
}

```

OUTPUT :

```

Press
1. push
2. pop
3. peek
4. print
5. exit
>>> 1
Enter the value : 10
Insertion successfull.
STACK TOP : 10
Press
1. push
2. pop
3. peek
4. print
5. exit
>>> 1
Enter the value : 20
Insertion successfull.
STACK TOP : 20 10
Press
1. push
2. pop
3. peek
4. print
5. exit
>>> 3
Peek : 20
STACK TOP : 20 10

```

```

Press
1. push
2. pop
3. peek
4. print
5. exit
>>> 4
STACK TOP : 20 10
Press
1. push
2. pop
3. peek
4. print
5. exit
>>> 2
Pop : 20
STACK TOP : 10
Press
1. push
2. pop
3. peek
4. print
5. exit
>>> 2
Pop : 10
STACK TOP :

```

```

Press
1. push
2. pop
3. peek
4. print
5. exit
>>> 2
Underflow.
STACK TOP :
Press
1. push
2. pop
3. peek
4. print
5. exit
>>> 5

```

QUESTION 7: Implement a system that can handle more than one stack (n stacks).

SOURCE CODE :

```
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>

#define MAX 10
#define STACK_COUNT 10

typedef struct stack {
    int stack[MAX];
    int top;
} stack;

bool isEmpty(stack *s) {
    return s->top == -1;
}

bool isFull(stack *s) {
    return s->top == MAX - 1;
}

void push(stack *s, int val) {
    if (isFull(s)) {
        printf("Overflow.\n");
        return;
    }

    s->stack[++(s->top)] = val;
    printf("Insertion successful.\n");
}

int pop(stack *s) {
    if (isEmpty(s)) {
        printf("Underflow.\n");
        return INT_MIN;
    }

    int val = s->stack[s->top];
    s->stack[s->top] = 0;
    s->top--;

    return val;
}

int peek(stack *s) {
    if (isEmpty(s)) {
        printf("Underflow.\n");
        return INT_MIN;
    }

    return s->stack[s->top];
}
```

```

void print(stack *s) {
    printf("STACK TOP : ");
    for (int i = s->top; i >= 0; i--) {
        printf(" %d ", s->stack[i]);
    }
    printf("\n");
}

void create(stack arr[], int n) {
    arr[n].top = -1;

    for (int i = 0; i < MAX; i++) {
        arr[n].stack[i] = 0;
    }
}

int main() {
    int ch, val;
    bool f = true;
    stack arr[STACK_COUNT];
    int size = 1;
    int n = 0;
    create(arr, 0);
    do {
        printf("\n===== \n");
        printf("Current Stack : %d\n", n);
        printf("Total Stacks : %d\n", size);
        printf("===== \n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Peek\n");
        printf("4. Print\n");
        printf("5. Create New Stack\n");
        printf("6. Switch Stack\n");
        printf("7. Exit\n");
        printf(">>> ");
        scanf("%d", &ch);

        switch (ch) {
            case 1:
                printf("Enter value : ");
                scanf("%d", &val);
                push(&arr[n], val);
                break;

            case 2:
                val = pop(&arr[n]);
                if (val != INT_MIN)
                    printf("Popped : %d\n", val);
                break;

            case 3:
                val = peek(&arr[n]);

```

```

        if (val != INT_MIN)
            printf("Top Element : %d\n", val);
        break;

    case 4:
        print(&arr[n]);
        break;

    case 5:
        if (size >= STACK_COUNT) {
            printf("Maximum stack limit reached.\n");
            break;
        }
        create(arr, size);
        printf("Stack %d created successfully.\n", size);
        size++;
        n = size - 1;
        break;

    case 6:
        printf("Available stacks: 0 to %d\n", size - 1);
        printf("Enter stack number : ");
        scanf("%d", &val);
        if (val >= 0 && val < size) {
            n = val;
            printf("Switched to stack %d.\n", n);
        } else {
            printf("Invalid stack number.\n");
        }
        break;

    case 7:
        f = false;
        break;

    default:
        printf("Wrong Input.\n");
        int c;
        while ((c = getchar()) != '\n' && c != EOF);

        break;
    }
} while (f);
printf("Program terminated.\n");
return 0;
}

```


OUTPUT :

```
=====
Current Stack : 0
Total Stacks : 1
=====
1. Push
2. Pop
3. Peek
4. Print
5. Create New Stack
6. Switch Stack
7. Exit
>>> 1
Enter value : 10
Insertion successful.

=====
Current Stack : 0
Total Stacks : 1
=====
1. Push
2. Pop
3. Peek
4. Print
5. Create New Stack
6. Switch Stack
7. Exit
>>> 1
Enter value : 20
Insertion successful.

=====
Current Stack : 0
Total Stacks : 1
=====
1. Push
2. Pop
3. Peek
4. Print
5. Create New Stack
6. Switch Stack
7. Exit
>>> 3
Top Element : 20

=====
Current Stack : 1
Total Stacks : 2
=====
1. Push
2. Pop
3. Peek
4. Print
5. Create New Stack
6. Switch Stack
7. Exit
>>> 1
Enter value : 10
Insertion successful.

=====
Current Stack : 1
Total Stacks : 2
=====
1. Push
2. Pop
3. Peek
4. Print
5. Create New Stack
6. Switch Stack
7. Exit
>>> 2
Popped : 30

=====
Current Stack : 1
Total Stacks : 2
=====
1. Push
2. Pop
3. Peek
4. Print
5. Create New Stack
6. Switch Stack
7. Exit
>>> 6
Available stacks: 0 to 1
Enter stack number : 0
Switched to stack 0.

=====
Current Stack : 0
Total Stacks : 1
=====
1. Push
2. Pop
3. Peek
4. Print
5. Create New Stack
6. Switch Stack
7. Exit
>>> 4
STACK TOP : 20 10

=====
Current Stack : 1
Total Stacks : 2
=====
1. Push
2. Pop
3. Peek
4. Print
5. Create New Stack
6. Switch Stack
7. Exit
>>> 1
Enter value : 30
Insertion successful.

=====
Current Stack : 1
Total Stacks : 2
=====
1. Push
2. Pop
3. Peek
4. Print
5. Create New Stack
6. Switch Stack
7. Exit
>>> 4
STACK TOP : 30 10
```

```
=====
Current Stack : 0
Total Stacks  : 2
=====
1. Push
2. Pop
3. Peek
4. Print
5. Create New Stack
6. Switch Stack
7. Exit
>>> 7
Program terminated.
```

QUESTION 8 : Print the data from a file of integers in reverse order using a stack.

SOURCE CODE :

```
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>
#define MAX 30

int stack[MAX];
int top = -1;

bool isEmpty() {
    return top == -1;
}

bool isFull() {
    return top == MAX - 1;
}

void push(int val) {
    if (isFull()) {
        printf("Overflow.\n");
        return;
    }
    top++;
    stack[top] = val;
    printf("Insertion successfull.\n");
}

int pop() {
    if (isEmpty()) {
        printf("Underflow.\n");
        return 0;
    }
    int val = stack[top];
    stack[top] = 0;
    top--;
    return val;
}

int peek() {
    if (isEmpty()) {
        printf("Underflow.\n");
        return 0;
    }
    return stack[top];
}

void print() {
    for (int i = MAX - 1; i >= 0; i--) {
        printf("| %d |\n-----\n", stack[i]);
    }
}
```

```

int main() {
    FILE *fp;
    int num;

    fp = fopen("data.txt", "r");

    if (fp == NULL) {
        printf("Cannot open file.\n");
        return 1;
    }

    while (fscanf(fp, "%d", &num) == 1) {
        push(num);
    }

    fclose(fp);

    printf("Numbers in reverse order:\n");

    while (!isEmpty()) {
        printf("%d ", pop());
    }

    printf("\n");

    return 0;
}

```

OUTPUT :

 data.txt		Insertion successfull.
1	10	Insertion successfull.
2	20	Insertion successfull.
3	30	Insertion successfull.
4	40	Insertion successfull.
5	50	Insertion successfull.
6	60	Insertion successfull.
7	70	Insertion successfull.
8	80	Insertion successfull.
9	90	Insertion successfull.
10	100	Numbers in reverse order:
		100 90 80 70 60 50 40 30 20 10

QUESTION 9: Write a Boolean function to return true if two stacks are equal.

SOURCE CODE :

```
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>

#define MAX 10
#define STACK_COUNT 10

typedef struct stack {
    int stack[MAX];
    int top;
} stack;

bool isEmpty(stack *s) {
    return s->top == -1;
}

bool isFull(stack *s) {
    return s->top == MAX - 1;
}

void push(stack *s, int val) {
    if (isFull(s)) {
        printf("Overflow.\n");
        return;
    }

    s->stack[++(s->top)] = val;
    printf("Insertion successful.\n");
}

int pop(stack *s) {
    if (isEmpty(s)) {
        printf("Underflow.\n");
        return INT_MIN;
    }

    int val = s->stack[s->top];
    s->stack[s->top] = 0;
    s->top--;

    return val;
}

int peek(stack *s) {
    if (isEmpty(s)) {
        printf("Underflow.\n");
        return INT_MIN;
    }

    return s->stack[s->top];
}
```

```

void print(stack *s) {
    printf("STACK TOP : ");
    for (int i = s->top; i >= 0; i--) {
        printf(" %d ", s->stack[i]);
    }
    printf("\n");
}

void create(stack arr[], int n) {
    arr[n].top = -1;

    for (int i = 0; i < MAX; i++) {
        arr[n].stack[i] = 0;
    }
}

bool isEqual(stack *s1, stack *s2) {
    if (s1->top != s2->top) return false;
    for (int i = 0; i <= s1->top; i++) {
        if (s1->stack[i] != s2->stack[i]) return false;
    }
    return true;
}

int main() {
    int ch, val;
    bool f = true;

    stack arr[STACK_COUNT];

    int size = 1;
    int n = 0;
    int i, j;

    create(arr, 0);

    do {
        printf("\n===== \n");
        printf("Current Stack : %d\n", n);
        printf("Total Stacks : %d\n", size);
        printf("===== \n");

        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Peek\n");
        printf("4. Print\n");
        printf("5. Create New Stack\n");
        printf("6. Switch Stack\n");
        printf("7. isEqual\n");
        printf("8. Exit\n");
        printf(">>> ");

        scanf("%d", &ch);
    } while (ch != 8);
}

```

```

switch (ch) {

case 1:
    printf("Enter value : ");
    scanf("%d", &val);

    push(&arr[n], val);
    break;

case 2:
    val = pop(&arr[n]);

    if (val != INT_MIN)
        printf("Popped : %d\n", val);

    break;

case 3:
    val = peek(&arr[n]);

    if (val != INT_MIN)
        printf("Top Element : %d\n", val);

    break;

case 4:
    print(&arr[n]);
    break;

case 5:
    if (size >= STACK_COUNT) {
        printf("Maximum stack limit reached.\n");
        break;
    }

    create(arr, size);

    printf("Stack %d created successfully.\n", size);

    size++;
    n = size - 1;
    break;

case 6:
    printf("Available stacks: 0 to %d\n", size - 1);
    printf("Enter stack number : ");
    scanf("%d", &val);

    if (val >= 0 && val < size) {
        n = val;
        printf("Switched to stack %d.\n", n);
    } else {
        printf("Invalid stack number.\n");
    }
}

```

```

        break;

    case 7:
        printf("Enter the stack numbers (i j) : ");
        scanf("%d %d",&i, &j);
        if (i >= 0 && j >= 0 && i <= n && j <= n) {
            if (isEqual(&arr[i], &arr[j])) {
                printf("Stacks are equal.\n");
            } else {
                printf("Stacks are unequal.\n");
            }
        } else {
            printf("wrong input.\n");
        }
        break;
    case 8:
        f = false;
        break;

    default:
        printf("Wrong Input.\n");

        int c;
        while ((c = getchar()) != '\n' && c != EOF);

        break;
}

} while (f);

printf("Program terminated.\n");

return 0;
}

```


OUTPUT :

```
=====
Current Stack : 0
Total Stacks : 1
=====
1. Push
2. Pop
3. Peek
4. Print
5. Create New Stack
6. Switch Stack
7. isEqual
8. Exit
>>> 1
Enter value : 10
Insertion successful.

=====
Current Stack : 0
Total Stacks : 1
=====
1. Push
2. Pop
3. Peek
4. Print
5. Create New Stack
6. Switch Stack
7. isEqual
8. Exit
>>> 1
Enter value : 20
Insertion successful.

=====
Current Stack : 0
Total Stacks : 1
=====
1. Push
2. Pop
3. Peek
4. Print
5. Create New Stack
6. Switch Stack
7. isEqual
8. Exit
>>> 5
Stack 1 created successfully.

=====
Current Stack : 1
Total Stacks : 2
=====
1. Push
2. Pop
3. Peek
4. Print
5. Create New Stack
6. Switch Stack
7. isEqual
8. Exit
>>> 1
Enter value : 10
Insertion successful.
```

```

=====
Current Stack : 1
Total Stacks : 2
=====
1. Push
2. Pop
3. Peek
4. Print
5. Create New Stack
6. Switch Stack
7. isEqual
8. Exit
>>> 2
Popped : 20

```

```

=====
Current Stack : 1
Total Stacks : 2
=====
1. Push
2. Pop
3. Peek
4. Print
5. Create New Stack
6. Switch Stack
7. isEqual
8. Exit
>>> 7
Enter the stack numbers (i j) : 0 1
Stacks are unequal.

```

```

=====
Current Stack : 1
Total Stacks : 2
=====
1. Push
2. Pop
3. Peek
4. Print
5. Create New Stack
6. Switch Stack
7. isEqual
8. Exit
>>> 1
Enter value : 20
Insertion successful.

```

```

=====
Current Stack : 1
Total Stacks : 2
=====
1. Push
2. Pop
3. Peek
4. Print
5. Create New Stack
6. Switch Stack
7. isEqual
8. Exit
>>> 7
Enter the stack numbers (i j) : 0 1
Stacks are equal.

```

QUESTION 10: Write a program for dynamic implementation (using a link list) of stacks (n stacks).

SOURCE CODE :

```

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

typedef struct Node
{
    int val;
    struct Node *next;
} Node;

Node *top = NULL;

void push(int val)
{
    Node *newNode = (Node *)malloc(sizeof(Node));

    if (newNode == NULL)
    {
        printf("Memory Allocation Failed.\n");
        return;
    }

```

```

    newNode->val = val;
    newNode->next = top;
    top = newNode;
}

Node *pop()
{
    if (top == NULL)
    {
        printf("Stack Underflow.\n");
        return NULL;
    }

    Node *temp = top;
    top = top->next;
    temp->next = NULL;

    return temp;
}

Node *peek()
{
    return top;
}

void display()
{
    if (top == NULL)
    {
        printf("Stack Empty.\n");
        return;
    }

    Node *temp = top;

    printf("Stack TOP = ");
    while (temp != NULL)
    {
        printf(" %d ", temp->val);
        temp = temp->next;
    }
    printf("\n");
}

void freeStack()
{
    Node *temp;

    while (top != NULL)

```

```

    {
        temp = top;
        top = top->next;
        free(temp);
    }
}

int main()
{
    int ch, val;
    bool running = true;
    Node *t;

    while (running)
    {
        printf("\n===== STACK USING LINKED LIST =====\n");
        printf("1. Push\n");
        printf("2. Pop\n");
        printf("3. Peek\n");
        printf("4. Display\n");
        printf("5. Exit\n");
        printf("Enter Choice : ");

        if (scanf("%d", &ch) != 1)
        {
            printf("Invalid Input.\n");

            int c;
            while ((c = getchar()) != '\n' && c != EOF)
                ;

            continue;
        }

        switch (ch)
        {
        case 1:
            printf("Enter Value : ");
            scanf("%d", &val);

            push(val);

            printf("After Push:\n");
            display();
            break;

        case 2:
            t = pop();

            if (t != NULL)
            {
                printf("Popped Element : %d\n", t->val);
                free(t);
            }

```

```

        break;

    case 3:
        t = peek();

        if (t != NULL)
            printf("Top Element : %d\n", t->val);
        else
            printf("Stack Empty.\n");

        break;

    case 4:
        display();
        break;

    case 5:
        running = false;
        freeStack();
        printf("Program Terminated.\n");
        break;

    default:
        printf("Wrong Choice.\n");
        int c;
        while ((c = getchar()) != '\n' && c != EOF);
    }
}

return 0;
}

```

OUTPUT :

===== STACK USING LINKED LIST =====

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter Choice : 1
Enter Value : 10
After Push:
Stack TOP = 10

===== STACK USING LINKED LIST =====

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter Choice : 1
Enter Value : 20
After Push:
Stack TOP = 20 10

===== STACK USING LINKED LIST =====

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter Choice : 1
Enter Value : 30
After Push:
Stack TOP = 30 20 10

===== STACK USING LINKED LIST =====

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter Choice : 3
Top Element : 30

===== STACK USING LINKED LIST =====

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter Choice : 2
Popped Element : 30

===== STACK USING LINKED LIST =====

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter Choice : 2
Popped Element : 20

===== STACK USING LINKED LIST =====

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter Choice : 2
Popped Element : 10

===== STACK USING LINKED LIST =====

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter Choice : 2
Stack Underflow.

===== STACK USING LINKED LIST =====

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter Choice : 5
Program Terminated.

QUESTION 11: Using a stack, write a program to convert an infix expression into its equivalent postfix expression.

SOURCE CODE :

```
#include <stdio.h>
#include <stdbool.h>
#include <ctype.h>

#define MAX 30

char stack[MAX];
int top = -1;

bool isEmpty()
{
    return top == -1;
}

bool isFull()
{
    return top == MAX - 1;
}

void push(char val)
{
    if (isFull())
    {
        printf("Stack Overflow.\n");
        return;
    }

    stack[++top] = val;
}

char pop()
{
    if (isEmpty())
    {
        printf("Stack Underflow.\n");
        return '\0';
    }

    return stack[top--];
}

char peek()
{
    if (isEmpty())
    {
        return '\0';
    }

    return stack[top];
}
```

```

int precedence(char op)
{
    switch (op)
    {
        case '^':
            return 3;

        case '*':
        case '/':
        case '%':
            return 2;

        case '+':
        case '-':
            return 1;

        default:
            return 0;
    }
}

void infixToPostfix(char infix[], char postfix[])
{
    int i = 0;
    int j = 0;
    char ch;

    while ((ch = infix[i]) != '\0') {
        if (isalnum(ch)) {
            postfix[j++] = ch;
        } else if (ch == '(') {
            push(ch);
        } else if (ch == ')') {
            while (!isEmpty() && peek() != '(') {
                postfix[j++] = pop();
            }

            if (!isEmpty() && peek() == '(') {
                pop();
            }
        } else {
            while (!isEmpty() && precedence(peek()) >= precedence(ch)) {
                postfix[j++] = pop();
            }

            push(ch);
        }

        i++;
    }

    while (!isEmpty()) {
        postfix[j++] = pop();
    }
}

```



```

    }

    postfix[j] = '\0';
}
int main(){
    char infix[100];
    char postfix[100];

    printf("Enter Infix Expression: ");
    scanf("%s", infix);

    infixToPostfix(infix, postfix);

    printf("Infix   : %s\n", infix);
    printf("Postfix  : %s\n", postfix);

    return 0;
}

```

OUTPUT :

```

Enter Infix Expression: ((A+B)*(C-D)/E)-F
Infix   : ((A+B)*(C-D)/E)-F
Postfix  : AB+CD-*E/F-

```

QUESTION 12: Write a program to evaluate postfix expression using a stack.

SOURCE CODE :

```
#include <stdio.h>
#include <ctype.h>
#define MAX 30
int stack[MAX];
int top = -1;
int isEmpty(){ return top == -1; }
int isFull(){ return top == MAX - 1; }
void push(int val){
    if (isFull()) {
        printf("Stack Overflow\n");
        return;
    }
    stack[++top] = val;
}
int pop(){
    if (isEmpty()) {
        printf("Stack Underflow\n");
        return -1;
    }
    return stack[top--];
}
int evaluatePostfix(char exp[]) {
    int i = 0;
    while (exp[i] != '\0') {
        char ch = exp[i];
        if (isdigit(ch)) { push(ch - '0'); }
        else if (ch == '+' || ch == '-' || ch == '*' || ch == '/') {
            int val1 = pop();
            int val2 = pop();
            switch (ch) {
                case '+': push(val2 + val1); break;
                case '-': push(val2 - val1); break;
                case '*': push(val2 * val1); break;
                case '/': push(val2 / val1); break;
            }
        }
        i++;
    }
    return pop();
}
int main() {
    char postfix[100];
    printf("Enter Postfix Expression: ");
    scanf("%s", postfix);
    int result = evaluatePostfix(postfix);
    printf("Result = %d\n", result);
    return 0;
}
```

OUTPUT :

Enter Postfix Expression: 53+2*
Result = 16

QUESTION 13: Write a program to check balanced brackets of an expression using stack.

SOURCE CODE :

```
#include <stdio.h>
#include <stdbool.h>

#define MAX 100

char stack[MAX];
int top = -1;

bool isEmpty()
{
    return top == -1;
}

bool isFull()
{
    return top == MAX - 1;
}

void push(char ch)
{
    if (isFull())
    {
        printf("Stack Overflow\n");
        return;
    }

    stack[++top] = ch;
}

char pop()
{
    if (isEmpty())
    {
        return '\0';
    }

    return stack[top--];
}

char peek()
{
    if (isEmpty())
        return '\0';

    return stack[top];
}

bool isMatching(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '{' && close == '}') ||
           (open == '[' && close == ']');
}
```

```

bool isBalanced(char exp[])
{
    int i = 0;

    while (exp[i] != '\0')
    {
        char ch = exp[i];

        if (ch == '(' || ch == '{' || ch == '[')
        {
            push(ch);
        }
        else if (ch == ')' || ch == '}' || ch == ']')
        {
            if (isEmpty())
                return false;

            char open = pop();

            if (!isMatching(open, ch))
                return false;
        }

        i++;
    }

    return isEmpty();
}

int main()
{
    char exp[100];

    printf("Enter Expression: ");
    scanf("%s", exp);

    if (isBalanced(exp))
        printf("Balanced Expression\n");
    else
        printf("Not Balanced Expression\n");

    return 0;
}

```

OUTPUT :

- PS C:\Users\sneha\Desktop
Enter Expression: (){[]}
Balanced Expression
- PS C:\Users\sneha\Desktop
Enter Expression: (())
Not Balanced Expression
- PS C:\Users\sneha\Desktop

QUESTION 14:

- Write a program to implement queue data structure using an array.
- Implement a system that can handle multiple queues (n queues).
- Append a queue x at the end of a queue y.
- A Boolean function to return true if two eq(q1, q2) are equal.
- Reverse a queue.
- Procedure replace(p, e, x) to replace every occurrence of element e in queue p with the value of x.

SOURCE CODE :

```
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>

#define MAX 10
#define QUEUE_COUNT 10

typedef struct Queue
{
    int queue[MAX];
    int front;
    int rear;
} Queue;

bool isEmpty(Queue *q)
{
    return q->front == -1;
}

bool isFull(Queue *q)
{
    return q->rear == MAX - 1;
}

void enqueue(Queue *q, int val)
{
    if (isFull(q))
    {
        printf("Overflow.\n");
        return;
    }

    if (isEmpty(q))
        q->front = 0;

    q->queue[++q->rear] = val;
}

int dequeue(Queue *q)
{
    if (isEmpty(q))
    {
        printf("Underflow.\n");
        return INT_MIN;
    }
}
```

```

    }

    int val = q->queue[q->front];

    if (q->front == q->rear)
        q->front = q->rear = -1;
    else
        q->front++;

    return val;
}

int peek(Queue *q)
{
    if (isEmpty(q))
        return INT_MIN;

    return q->queue[q->front];
}

void printQueue(Queue *q)
{
    if (isEmpty(q))
    {
        printf("Queue Empty.\n");
        return;
    }

    printf("Front = ");

    for (int i = q->front; i <= q->rear; i++)
        printf("%d ", q->queue[i]);

    printf("= Rear\n");
}

void createQueue(Queue arr[], int n)
{
    arr[n].front = -1;
    arr[n].rear = -1;

    for (int i = 0; i < MAX; i++)
        arr[n].queue[i] = 0;
}

void appendQueue(Queue *y, Queue *x)
{
    while (!isEmpty(x))
    {
        if (isFull(y))
        {
            printf("Queue Y Overflow.\n");

```

```

        return;
    }

    enqueue(y, dequeue(x));
}

printf("Queue Appended Successfully.\n");
}

```

```

bool eq(Queue *q1, Queue *q2)
{
    int size1 = 0;
    int size2 = 0;

    if (isEmpty(q1) && isEmpty(q2))
        return true;

    if (isEmpty(q1) || isEmpty(q2))
        return false;

    size1 = q1->rear - q1->front + 1;
    size2 = q2->rear - q2->front + 1;

    if (size1 != size2)
        return false;

    int i = q1->front;
    int j = q2->front;

    while (i <= q1->rear)
    {
        if (q1->queue[i] != q2->queue[j])
            return false;

        i++;
        j++;
    }

    return true;
}

```

```

void reverseQueue(Queue *q)
{
    if (isEmpty(q))
    {
        printf("Queue Empty.\n");
        return;
    }

    int temp[MAX];
    int count = 0;

    for (int i = q->front; i <= q->rear; i++)

```



```

        temp[count++] = q->queue[i];

    int left = 0;
    int right = count - 1;

    while (left < right)
    {
        int t = temp[left];
        temp[left] = temp[right];
        temp[right] = t;

        left++;
        right--;
    }

    q->front = 0;
    q->rear = count - 1;

    for (int i = 0; i < count; i++)
        q->queue[i] = temp[i];

    printf("Queue Reversed Successfully.\n");
}

void replace(Queue *q, int e, int x) {
    if (isEmpty(q)) {
        printf("Queue Empty.\n");
        return;
    }

    for (int i = q->front; i <= q->rear; i++) {
        if (q->queue[i] == e) {
            q->queue[i] = x;
        }
    }
}

int main()
{
    Queue arr[QUEUE_COUNT];

    int size = 1;
    int current = 0;

    createQueue(arr, 0);

    int ch, val;
    bool running = true;

    while (running)
    {
        printf("\n===== \n");
        printf("Current Queue : %d\n", current);
    }
}

```

```

printf("Total Queues : %d\n", size);
printf("=====\n");

printf("1. Enqueue\n");
printf("2. Dequeue\n");
printf("3. Peek\n");
printf("4. Print\n");
printf("5. Create Queue\n");
printf("6. Switch Queue\n");
printf("7. Append Queue X to Y\n");
printf("8. Compare Two Queues\n");
printf("9. Reverse Queue\n");
printf("10. Replace\n");
printf("11. Exit\n");
printf(">>> ");

scanf("%d", &ch);

switch (ch)
{
    case 1:
        printf("Value : ");
        scanf("%d", &val);
        enqueue(&arr[current], val);
        break;

    case 2:
        val = dequeue(&arr[current]);

        if (val != INT_MIN)
            printf("Deleted : %d\n", val);

        break;

    case 3:
        val = peek(&arr[current]);

        if (val != INT_MIN)
            printf("Front : %d\n", val);

        break;

    case 4:
        printQueue(&arr[current]);
        break;

    case 5:
        if (size >= QUEUE_COUNT)
        {
            printf("Maximum Queue Limit Reached.\n");
            break;
        }

        createQueue(arr, size);

```

```

        printf("Queue %d Created.\n", size);

        size++;
        break;

    case 6:
        printf("Enter Queue Number (0-%d): ", size - 1);
        scanf("%d", &val);

        if (val >= 0 && val < size)
            current = val;
        else
            printf("Invalid Queue Number.\n");

        break;

    case 7:
    {
        int x, y;

        printf("Source Queue X : ");
        scanf("%d", &x);

        printf("Destination Queue Y : ");
        scanf("%d", &y);

        if (x >= 0 && x < size &&
            y >= 0 && y < size)
        {
            appendQueue(&arr[y], &arr[x]);
        }

        break;
    }

    case 8:
    {
        int q1, q2;
        printf("Queue 1 : ");
        scanf("%d", &q1);
        printf("Queue 2 : ");
        scanf("%d", &q2);
        if (eq(&arr[q1], &arr[q2]))
            printf("Queues are Equal.\n");
        else
            printf("Queues are Not Equal.\n");
        break;
    }

    case 9:
        reverseQueue(&arr[current]);
        break;

```

```

        case 10:
        {
            int p, e, x;
            printf("Enter the queue : ");
            scanf("%d",&p);
            printf("Enter the value : ");
            scanf("%d",&e);
            printf("Enter the new value : ");
            scanf("%d",&x);
            replace(&arr[p], e, x);
            break;
        }
        case 11:
            running = false;
            break;
        default:
            printf("Wrong Input.\n");
    }
}
return 0;
}

```

OUTPUT :

```

=====
Current Queue : 0
Total Queues : 1
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 1
Value : 10

```

```

=====
Current Queue : 0
Total Queues : 1
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 1
Value : 20

```

```

=====
Current Queue : 0
Total Queues : 1
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 4
Front = 10 20 = Rear

```

```

=====
Current Queue : 0
Total Queues : 1
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 5
Queue 1 Created.

```

```

=====
Current Queue : 0
Total Queues : 2
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 6
Enter Queue Number (0-1): 1

```

```

=====
Current Queue : 1
Total Queues : 2
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 1
Value : 10

```

=====	=====	=====
Current Queue : 1	Current Queue : 1	Current Queue : 1
Total Queues : 2	Total Queues : 2	Total Queues : 2
=====	=====	=====
1. Enqueue	1. Enqueue	1. Enqueue
2. Dequeue	2. Dequeue	2. Dequeue
3. Peek	3. Peek	3. Peek
4. Print	4. Print	4. Print
5. Create Queue	5. Create Queue	5. Create Queue
6. Switch Queue	6. Switch Queue	6. Switch Queue
7. Append Queue X to Y	7. Append Queue X to Y	7. Append Queue X to Y
8. Compare Two Queues	8. Compare Two Queues	8. Compare Two Queues
9. Reverse Queue	9. Reverse Queue	9. Reverse Queue
10. Replace	10. Replace	10. Replace
11. Exit	11. Exit	11. Exit
>>> 1	>>> 8	>>> 4
Value : 20	Queue 1 : 1	Front = 10 20 10 20 = Rear
	Queue 2 : 0	
	Queues are Equal.	
=====	=====	=====
Current Queue : 1	Current Queue : 1	Current Queue : 1
Total Queues : 2	Total Queues : 2	Total Queues : 2
=====	=====	=====
1. Enqueue	1. Enqueue	1. Enqueue
2. Dequeue	2. Dequeue	2. Dequeue
3. Peek	3. Peek	3. Peek
4. Print	4. Print	4. Print
5. Create Queue	5. Create Queue	5. Create Queue
6. Switch Queue	6. Switch Queue	6. Switch Queue
7. Append Queue X to Y	7. Append Queue X to Y	7. Append Queue X to Y
8. Compare Two Queues	8. Compare Two Queues	8. Compare Two Queues
9. Reverse Queue	9. Reverse Queue	9. Reverse Queue
10. Replace	10. Replace	10. Replace
11. Exit	11. Exit	11. Exit
>>> 4	>>> 7	>>> 9
Front = 10 20 = Rear	Source Queue X : 0	Queue Reversed Successfully.
	Destination Queue Y : 1	
	Queue Appended Successfully.	

```

=====
Current Queue : 1
Total Queues : 2
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>>
4
Front = 20 10 20 10 = Rear

=====
Current Queue : 1
Total Queues : 2
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 10
Enter the queue : 1
Enter the value : 10
Enter the new value : 50

=====
Current Queue : 1
Total Queues : 2
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 4
Front = 20 50 20 50 = Rear

=====
Current Queue : 1
Total Queues : 2
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 11

```

QUESTION 15: Write a program for dynamic implementation (using a link list) of a queue of the above.

SOURCE CODE :

```

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define QUEUE_COUNT 10

typedef struct Node
{
    int data;
    struct Node *next;
} Node;

typedef struct Queue
{
    Node *front;
    Node *rear;
} Queue;

bool isEmpty(Queue *q)
{
    return q->front == NULL;
}

void enqueue(Queue *q, int val)

```

```

{
    Node *newNode = (Node *)malloc(sizeof(Node));

    if (newNode == NULL)
    {
        printf("Memory Allocation Failed.\n");
        return;
    }

    newNode->data = val;
    newNode->next = NULL;

    if (isEmpty(q))
    {
        q->front = q->rear = newNode;
    }
    else
    {
        q->rear->next = newNode;
        q->rear = newNode;
    }
}

int dequeue(Queue *q)
{
    if (isEmpty(q))
    {
        printf("Underflow.\n");
        return -1;
    }

    Node *temp = q->front;
    int val = temp->data;

    q->front = q->front->next;

    if (q->front == NULL)
        q->rear = NULL;

    free(temp);

    return val;
}

int peek(Queue *q)
{
    if (isEmpty(q))
    {
        printf("Queue Empty.\n");
        return -1;
    }

    return q->front->data;
}

```

```

void printQueue(Queue *q)
{
    if (isEmpty(q))
    {
        printf("Queue Empty.\n");
        return;
    }

    Node *temp = q->front;

    printf("Front = ");

    while (temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }

    printf("= Rear\n");
}

void createQueue(Queue arr[], int n)
{
    arr[n].front = NULL;
    arr[n].rear = NULL;
}

void appendQueue(Queue *y, Queue *x)
{
    if (isEmpty(x))
    {
        printf("Source Queue Empty.\n");
        return;
    }

    if (isEmpty(y))
    {
        y->front = x->front;
        y->rear = x->rear;
    }
    else
    {
        y->rear->next = x->front;
        y->rear = x->rear;
    }

    x->front = NULL;
    x->rear = NULL;

    printf("Queue Appended Successfully.\n");
}

bool eq(Queue *q1, Queue *q2)

```



```

{
    Node *p = q1->front;
    Node *q = q2->front;

    while (p != NULL && q != NULL)
    {
        if (p->data != q->data)
            return false;

        p = p->next;
        q = q->next;
    }

    return (p == NULL && q == NULL);
}

void reverseQueue(Queue *q)
{
    if (isEmpty(q))
        return;

    Node *prev = NULL;
    Node *curr = q->front;
    Node *next = NULL;

    q->rear = q->front;

    while (curr != NULL)
    {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }

    q->front = prev;

    printf("Queue Reversed Successfully.\n");
}

void replace(Queue *q, int e, int x)
{
    Node *temp = q->front;

    while (temp != NULL)
    {
        if (temp->data == e)
            temp->data = x;

        temp = temp->next;
    }

    printf("Replacement Done.\n");
}

```

```

int main()
{
    Queue arr[QUEUE_COUNT];

    int size = 1;
    int current = 0;

    createQueue(arr, 0);

    int ch, val;
    bool running = true;

    while (running)
    {
        printf("\n=====\\n");
        printf("Current Queue : %d\\n", current);
        printf("Total Queues : %d\\n", size);
        printf("=====\\n");

        printf("1. Enqueue\\n");
        printf("2. Dequeue\\n");
        printf("3. Peek\\n");
        printf("4. Print\\n");
        printf("5. Create Queue\\n");
        printf("6. Switch Queue\\n");
        printf("7. Append Queue X to Y\\n");
        printf("8. Compare Queues\\n");
        printf("9. Reverse Queue\\n");
        printf("10. Replace\\n");
        printf("11. Exit\\n");
        printf(">>> ");

        scanf("%d", &ch);

        switch (ch)
        {
            case 1:
                printf("Value : ");
                scanf("%d", &val);
                enqueue(&arr[current], val);
                break;

            case 2:
                printf("Deleted : %d\\n",
                    dequeue(&arr[current]));
                break;

            case 3:
                printf("Front : %d\\n",
                    peek(&arr[current]));
                break;

            case 4:

```

```

        printQueue(&arr[current]);
        break;

case 5:
    if (size >= QUEUE_COUNT)
    {
        printf("Maximum Queue Limit Reached.\n");
        break;
    }

    createQueue(arr, size);
    printf("Queue %d Created.\n", size);
    size++;
    break;

case 6:
    printf("Queue Number (0-%d): ",
           size - 1);
    scanf("%d", &val);

    if (val >= 0 && val < size)
        current = val;
    else
        printf("Invalid Queue.\n");

    break;

case 7:
{
    int x, y;

    printf("Source Queue X : ");
    scanf("%d", &x);

    printf("Destination Queue Y : ");
    scanf("%d", &y);

    if (x >= 0 && x < size &&
        y >= 0 && y < size)
    {
        appendQueue(&arr[y], &arr[x]);
    }

    break;
}

case 8:
{
    int q1, q2;

    printf("Queue 1 : ");
    scanf("%d", &q1);

    printf("Queue 2 : ");

```

```

        scanf("%d", &q2);

        if (eq(&arr[q1], &arr[q2]))
            printf("Queues are Equal.\n");
        else
            printf("Queues are Not Equal.\n");

        break;
    }

    case 9:
        reverseQueue(&arr[current]);
        break;

    case 10:
    {
        int p, e, x;

        printf("Queue Number : ");
        scanf("%d", &p);

        printf("Element : ");
        scanf("%d", &e);

        printf("New Value : ");
        scanf("%d", &x);

        replace(&arr[p], e, x);

        break;
    }

    case 11:
        running = false;
        break;

    default:
        printf("Wrong Input.\n");
    }
}

return 0;
}

```

OUTPUT :

```

=====
Current Queue : 0
Total Queues : 1
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 1
Value : 10

```

```

=====
Current Queue : 0
Total Queues : 1
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 1
Value : 20

```

```

=====
Current Queue : 0
Total Queues : 1
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 4
Front = 10 20 = Rear

```

```

=====
Current Queue : 0
Total Queues : 1
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 5
Queue 1 Created.

```

```

=====
Current Queue : 0
Total Queues : 2
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 6
Enter Queue Number (0-1): 1

```

```

=====
Current Queue : 1
Total Queues : 2
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 1
Value : 10

```

```
=====
Current Queue : 1
Total Queues : 2
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 1
Value : 20
```

```
=====
Current Queue : 1
Total Queues : 2
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 4
Front = 10 20 = Rear
```

```
=====
Current Queue : 1
Total Queues : 2
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 8
Queue 1 : 1
Queue 2 : 0
Queues are Equal.
```

```
=====
Current Queue : 1
Total Queues : 2
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 7
Source Queue X : 0
Destination Queue Y : 1
Queue Appended Successfully.
```

```
=====
Current Queue : 1
Total Queues : 2
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 4
Front = 10 20 10 20 = Rear
```

```
=====
Current Queue : 1
Total Queues : 2
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 9
Queue Reversed Successfully.
```

```

=====
Current Queue : 1
Total Queues : 2
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>>
4
Front = 20 10 20 10 = Rear

```

```

=====
Current Queue : 1
Total Queues : 2
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 10
Enter the queue : 1
Enter the value : 10
Enter the new value : 50

```

```

=====
Current Queue : 1
Total Queues : 2
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 4
Front = 20 50 20 50 = Rear

```

```

=====
Current Queue : 1
Total Queues : 2
=====
1. Enqueue
2. Dequeue
3. Peek
4. Print
5. Create Queue
6. Switch Queue
7. Append Queue X to Y
8. Compare Two Queues
9. Reverse Queue
10. Replace
11. Exit
>>> 11

```

QUESTION 16: Implement a circular queue using an array.

SOURCE CODE :

```

#include <stdio.h>
#include <stdbool.h>
#define MAX 5
int front = -1, rear = -1;
int queue[MAX];
bool isEmpty(){
    return front == -1;
}
bool isFull(){
    return (rear + 1) % MAX == front;
}
void enqueue(int val)
{
    if (isFull()) {
        printf("Overflow\n");
        return;
    }
    if (isEmpty()) {
        front = rear = 0;
    } else {
        rear = (rear + 1) % MAX;
    }
    queue[rear] = val;
    printf("Element Added\n");
}
int dequeue()

```

```

{
    if (isEmpty()) {
        printf("Underflow\n");
        return -1;
    }
    int val = queue[front];
    if (front == rear) {
        front = rear = -1;
    } else {
        front = (front + 1) % MAX;
    }
    return val;
}

int peek()
{
    if (isEmpty()) {
        printf("Queue Empty\n");
        return -1;
    }
    return queue[front];
}

void display()
{
    if (isEmpty()) {
        printf("Queue Empty\n");
        return;
    }
    printf("Front = ");
    int i = front;
    while (1) {
        printf("%d ", queue[i]);
        if (i == rear) break;
        i = (i + 1) % MAX;
    }

    printf("= Rear\n");
}

int main()
{
    int ch, val;
    bool running = true;
    while (running)
    {
        printf("\n-----\n");
        printf("          Circular Queue\n");
        printf("-----\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Peek\n");
        printf("4. Display\n");
        printf("5. Exit\n");
        printf(">>> ");
    }
}

```



```

scanf("%d", &ch);

switch (ch)
{
case 1:
    printf("Enter Value : ");
    scanf("%d", &val);

    enqueue(val);
    break;

case 2:
    val = dequeue();

    if (val != -1) printf("Dequeued : %d\n", val);
    break;

case 3:
    val = peek();

    if (val != -1) printf("Front Element : %d\n", val);
    break;
case 4:
    display();
    break;
case 5:
    running = false;
    break;
default:
    printf("Wrong Input\n");
    break;
}
}
return 0;
}

```

OUTPUT :

```

-----
Circular Queue
-----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
>>> 1
Enter Value : 10
Element Added

```

```

-----
Circular Queue
-----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
>>> 4
Front = 10 20 30 = Rear

```

```

-----
Circular Queue
-----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
>>> 2
Dequeued : 10

```

```

-----
Circular Queue
-----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
>>> 1
Enter Value : 20
Element Added

```

```

-----
Circular Queue
-----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
>>> 3
Front Element : 10

```

```

-----
Circular Queue
-----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
>>> 2
Dequeued : 20

```

```

-----
Circular Queue
-----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
>>> 1
Enter Value : 30
Element Added

```

```

-----
Circular Queue
-----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
>>> 2
Dequeued : 10

```

```

-----
Circular Queue
-----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
>>> 2
Dequeued : 30

```

```

-----
Circular Queue
-----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
>>> 2
Underflow

```

```

-----
Circular Queue
-----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
>>> 5

```

QUESTION 17: Implement a Deque using a doubly linked list.

SOURCE CODE :

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <stdbool.h>
typedef struct node {
    int val;
    struct node *prev;
    struct node *next;
}node;

```

```

node *front = NULL, *rear = NULL;
void eq(int val, int ch) {
    node *newNode = (node*)malloc(sizeof(node));
    if (newNode == NULL) {
        printf("Memory allocation failed. Retry\n");
        return;
    }
    newNode->val = val;
    if (front == NULL && rear == NULL) {
        newNode->next = NULL;
        newNode->prev = NULL;
        front = newNode;
        rear = newNode;
        return;
    }
    if (ch == 0) {
        newNode->next = front;
        front->prev = newNode;
        newNode->prev = NULL;
        front = newNode;
    } else {
        rear->next = newNode;
        newNode->prev = rear;
        newNode->next = NULL;
        rear = newNode;
    }
    printf("Enqueue done\n");
}
int dq(int ch) {
    if (front == NULL && rear == NULL) {
        printf("Underflow");
        return -1;
    }
    int val;
    if (ch == 0) {
        val = front->val;
        if (front->next == NULL) {
            front = NULL;
            rear = NULL;
        } else {
            front = front->next;
            front->prev = NULL;
        }
    } else {
        val = rear->val;
        if (rear->prev == NULL) {
            rear = NULL;
            front = NULL;
        } else {
            rear = rear->prev;
            rear->next = NULL;
        }
    }
    return val;
}

```

```

}

void dis(int ch) {
    if (front == NULL && rear == NULL) {
        printf("Queue is empty\n");
        return;
    }
    if (ch == 0) {
        node *temp = front;
        printf("Front = ");
        while (temp != NULL) {
            printf("[%p | %d | %p] -> ", temp->prev, temp->val, temp->next);
            temp = temp->next;
        }
        printf(" = Rear\n");
    } else {
        node *temp = rear;
        printf("Rear = ");
        while (temp != NULL) {
            printf("[%p | %d | %p] -> ", temp->next, temp->val, temp->prev);
            temp = temp->prev;
        }
        printf(" = Front\n");
    }
    printf("\n");
}

int main() {
    int ch, val;
    bool f = true;
    while (f) {
        printf("\n-----\n");
        printf("          Queue          \n");
        printf("-----\n");
        printf("1. enqueue\n");
        printf("2. dequeue\n");
        printf("3. exit\n>>> ", ch);
        scanf("%d",&ch);

        switch (ch) {
            case 1:
                val = (rand() % 99) + 1;
                eq(val, 0);
                dis(0);
                dis(1);
                break;
            case 2:
                val = dq(1);
                if (val != -1) {
                    printf("Removed : %d\n", val);
                    dis(0);
                    dis(1);
                }
                break;
        }
    }
}

```

```

        default:
            f = false;
            break;
    }
}
}

```

OUTPUT :

```

-----
Queue
-----
1. enqueue
2. dequeu
3. exit
>>> 1
Front = [00000000 | 42 | 00000000] -> = Rear

Rear = [00000000 | 42 | 00000000] -> = Front

```

```

-----
Queue
-----
1. enqueue
2. dequeu
3. exit
>>> 1
Enqueue done
Front = [00000000 | 54 | 00CA18B0] -> [00CA17E8 | 42 | 00000000] -> = Rear

Rear = [00000000 | 42 | 00CA17E8] -> [00CA18B0 | 54 | 00000000] -> = Front

```

```

-----
Queue
-----
1. enqueue
2. dequeu
3. exit
>>> 1
Enqueue done
Front = [00000000 | 98 | 00CA17E8] -> [00CA1800 | 54 | 00CA18B0] -> [00CA17E8 | 42 | 00000000] -> = Rear

Rear = [00000000 | 42 | 00CA17E8] -> [00CA18B0 | 54 | 00CA1800] -> [00CA17E8 | 98 | 00000000] -> = Front

```

```

-----
Queue
-----
1. enqueue
2. dequeu
3. exit
>>> 2
Removed : 42
Front = [00000000 | 98 | 00CA17E8] -> [00CA1800 | 54 | 00000000] -> = Rear

Rear = [00000000 | 54 | 00CA1800] -> [00CA17E8 | 98 | 00000000] -> = Front

```

```

-----
Queue
-----
1. enqueue
2. dequeu
3. exit
>>> 2
Removed : 54
Front = [00000000 | 98 | 00000000] -> = Rear

Rear = [00000000 | 98 | 00000000] -> = Front

```

```

-----
Queue
-----
1. enqueue
2. dequeu
3. exit
>>> 2
Removed : 98
Queue is empty
Queue is empty

```

```

-----
Queue
-----
1. enqueue
2. dequeu
3. exit
>>> 3

```

QUESTION 18: Implement a priority queue using

a. A single array.

SOURCE CODE :

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <stdbool.h>
#define MAX 10
typedef struct{
    int val;

```

```

    int priority;
} Node;
Node queue[MAX];
int front = -1;
int rear = -1;

bool isEmpty(){
    return front == -1;
}
bool isFull(){
    return rear == MAX - 1;
}
void enqueue(int val, int priority){
    if (isFull()) {
        printf("Overflow\n");
        return;
    }
    if (isEmpty()) {
        front = rear = 0;
    } else {
        rear++;
    }
    queue[rear].val = val;
    queue[rear].priority = priority;
    printf("Enqueued Successfully.\n");
}
Node peek(){
    Node temp = {-1, -1};
    if (isEmpty()){
        printf("Underflow\n");
        return temp;
    }
    int maxIndex = front;
    for (int i = front + 1; i <= rear; i++) {
        if (queue[i].priority > queue[maxIndex].priority) {
            maxIndex = i;
        }
    }
    return queue[maxIndex];
}
Node dequeue() {
    Node temp = {-1, -1};
    if (isEmpty()) {
        printf("Underflow\n");
        return temp;
    }
    int maxIndex = front;
    for (int i = front + 1; i <= rear; i++) {
        if (queue[i].priority > queue[maxIndex].priority) {
            maxIndex = i;
        }
    }
    temp = queue[maxIndex];
    for (int i = maxIndex; i < rear; i++) {

```

```

        queue[i] = queue[i + 1];
    }
    if (front == rear) {
        front = rear = -1;
    } else {
        rear--;
    }
    return temp;
}

void display(){
    if (isEmpty()) {
        printf("Queue Empty\n");
        return;
    }

    printf("Front = ");

    for (int i = front; i <= rear; i++)
    {
        printf("[%d|%d] ",
                queue[i].val,
                queue[i].priority);
    }

    printf("= Rear\n");
}

int main()
{
    srand(time(NULL));
    int ch;
    int val;
    int priority;
    Node temp;

    while (1)
    {
        printf("\n-----\n");
        printf("        Priority Queue\n");
        printf("-----\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Peek\n");
        printf("4. Exit\n");
        printf(">>> ");
        scanf("%d",&ch);

        switch (ch)
        {
            case 1:
                val = rand() % 100;
                priority = rand() % 10;
                printf("Enter Value : %d\n", val);

```

```

        printf("Enter Priority : %d\n", priority);

        enqueue(val, priority);
        display();
        break;

case 2:
    temp = dequeue();

    if (temp.val != -1)
    {
        printf("Dequeued : [%d | %d]\n",
            temp.val,
            temp.priority);
    }
    display();
    break;

case 3:
    temp = peek();

    if (temp.val != -1)
    {
        printf("Highest Priority Element : [%d | %d]\n",
            temp.val,
            temp.priority);
    }
    display();
    break;

default:
    printf("Program terminating ... \n");
    return 0;
}
}
}

```

OUTPUT :

Priority Queue

```
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 1
Enter Value : 29
Enter Priority : 0
Enqueued Successfully.
Front = [29|0] = Rear
```

Priority Queue

```
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 1
Enter Value : 85
Enter Priority : 1
Enqueued Successfully.
Front = [29|0] [85|1] = Rear
```

Priority Queue

```
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 1
Enter Value : 72
Enter Priority : 3
Enqueued Successfully.
Front = [29|0] [85|1] [72|3] = Rear
```

Priority Queue

```
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 3
Highest Priority Element : [72 | 3]
Front = [29|0] [85|1] [72|3] = Rear
```

Priority Queue

```
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 2
Dequeued : [72 | 3]
Front = [29|0] [85|1] = Rear
```

Priority Queue

```
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 2
Dequeued : [85 | 1]
Front = [29|0] = Rear
```

Priority Queue

```
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 2
Dequeued : [29 | 0]
Queue Empty
```

Priority Queue

```
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 2
Underflow
Queue Empty
```

Priority Queue

```
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 4
Program terminating ...
```

b. A single linked list.

SOURCE CODE :

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <stdbool.h>
typedef struct Node{
    int val;
    int priority;
    struct Node *next;
} Node;

Node *front = NULL;
bool isEmpty(){
    return front == NULL;
}

void enqueue(int val, int priority){
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("Overflow (Memory Allocation Failed)\n");
        return;
    }
    newNode->val = val;
    newNode->priority = priority;
    newNode->next = front;
    front = newNode;
    printf("Enqueued Successfully.\n");
}

Node peek() {
    Node temp = {-1, -1, NULL};

    if (isEmpty()) {
        printf("Underflow\n");
        return temp;
    }
    Node *ptr = front;
    Node *max = front;
    while (ptr != NULL) {
        if (ptr->priority > max->priority)
        {
            max = ptr;
        }
        ptr = ptr->next;
    }
    temp.val = max->val;
    temp.priority = max->priority;
    return temp;
}

Node dequeue() {
    Node temp = {-1, -1, NULL};
    if (isEmpty()) {
        printf("Underflow\n");
        return temp;
    }
```

```

    }
    Node *ptr = front;
    Node *prev = NULL;
    Node *max = front;
    Node *maxPrev = NULL;
    while (ptr != NULL) {
        if (ptr->priority > max->priority) {
            max = ptr;
            maxPrev = prev;
        }
        prev = ptr;
        ptr = ptr->next;
    }

    temp.val = max->val;
    temp.priority = max->priority;
    if (maxPrev == NULL) {
        front = front->next;
    } else {
        maxPrev->next = max->next;
    }
    free(max);
    return temp;
}

void display() {
    if (isEmpty()) {
        printf("Queue Empty\n");
        return;
    }
    Node *ptr = front;
    printf("Front = ");
    while (ptr != NULL) {
        printf("[%d|%d] ", ptr->val, ptr->priority);
        ptr = ptr->next;
    }
    printf("= Rear\n");
}

int main()
{
    srand(time(NULL));
    int ch;
    int val;
    int priority;
    Node temp;

    while (1) {
        printf("\n-----\n");
        printf("      Priority Queue (List)\n");
        printf("-----\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Peek\n");
        printf("4. Exit\n");
    }
}

```

```

printf(">>> ");
scanf("%d", &ch);

switch (ch)
{
case 1:
    val = rand() % 100;
    priority = rand() % 10;
    printf("Enter Value : %d\n", val);
    printf("Enter Priority : %d\n", priority);

    enqueue(val, priority);
    display();
    break;

case 2:
    temp = dequeue();

    if (temp.val != -1)
    {
        printf("Dequeued : [%d | %d]\n",
            temp.val,
            temp.priority);
    }
    display();
    break;

case 3:
    temp = peek();

    if (temp.val != -1)
    {
        printf("Highest Priority Element : [%d | %d]\n",
            temp.val,
            temp.priority);
    }
    display();
    break;

default:
    printf("Program terminating ... \n");
    return 0;
}
}
}

```

OUTPUT :

```
-----
Priority Queue (List)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 1
Enter Value : 45
Enter Priority : 5
Enqueued Successfully.
Front = [45|5] = Rear

-----
Priority Queue (List)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 1
Enter Value : 90
Enter Priority : 7
Enqueued Successfully.
Front = [90|7] [45|5] = Rear

-----
Priority Queue (List)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 1
Enter Value : 78
Enter Priority : 2
Enqueued Successfully.
Front = [78|2] [90|7] [45|5] = Rear

-----
Priority Queue (List)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 1
Enter Value : 45
Enter Priority : 5
Enqueued Successfully.
Front = [45|5] = Rear

-----
Priority Queue (List)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 3
Highest Priority Element : [90 | 7]
Front = [78|2] [90|7] [45|5] = Rear

-----
Priority Queue (List)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 2
Dequeued : [78 | 2]
Queue Empty

-----
Priority Queue (List)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 2
Underflow
Queue Empty

-----
Priority Queue (List)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 2
Dequeued : [90 | 7]
Front = [78|2] [45|5] = Rear

-----
Priority Queue (List)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 2
Dequeued : [45 | 5]
Front = [78|2] = Rear

-----
Priority Queue (List)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>>
4
Program terminating ...
```

c. A 2D array.

SOURCE CODE :

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <stdbool.h>

#define PRIORITY 10
#define SIZE 10

int queue[PRIORITY][SIZE];
int rear[PRIORITY];

void init() {
    for(int i = 0; i < PRIORITY; i++)
        rear[i] = -1;
}

bool isEmpty() {
    for(int i = 0; i < PRIORITY; i++)
        if(rear[i] != -1)
            return false;

    return true;
}

void enqueue(int val, int priority) {
    if (priority < 0 || priority >= PRIORITY) {
        printf("Invalid Priority Level\n");
        return;
    }

    if(rear[priority] == SIZE - 1) {
        printf("Overflow\n");
        return;
    }

    rear[priority]++;
    queue[priority][rear[priority]] = val;

    printf("Enqueued Successfully.\n");
}

int peek() {
    if(isEmpty()) {
        printf("Underflow\n");
        return -1;
    }

    for(int i = PRIORITY - 1; i >= 0; i--) {
        if(rear[i] != -1)
            return queue[i][0];
    }
}
```

```

    return -1;
}

int dequeue() {
    if(isEmpty()) {
        printf("Underflow\n");
        return -1;
    }

    for(int i = PRIORITY - 1; i >= 0; i--) {
        if(rear[i] != -1) {
            int temp = queue[i][0];

            for(int j = 0; j < rear[i]; j++)
                queue[i][j] = queue[i][j + 1];

            rear[i]--;

            return temp;
        }
    }

    return -1;
}

void display() {
    if(isEmpty()) {
        printf("Queue Empty\n");
        return;
    }

    printf("\n--- Current Queue State ---");
    for(int i = PRIORITY - 1; i >= 0; i--) {
        if (rear[i] != -1) {
            printf("\nPriority %d : ", i);
            for(int j = 0; j <= rear[i]; j++)
                printf("[%d] ", queue[i][j]);
        }
    }
    printf("\n-----\n");
}

int main()
{
    srand(time(NULL));
    init();

    int ch;
    int val;
    int priority;
    int temp;

    while (1)
    {

```

```

printf("\n-----\n");
printf("  Priority Queue (2D Array)\n");
printf("-----\n");
printf("1. Enqueue\n");
printf("2. Dequeue\n");
printf("3. Peek\n");
printf("4. Exit\n");
printf(">>> ");
scanf("%d", &ch);

switch (ch)
{
case 1:
    val = rand() % 100;
    priority = rand() % 10;
    printf("Enter Value : %d\n", val);
    printf("Enter Priority : %d\n", priority);

    enqueue(val, priority);
    display();
    break;

case 2:
    temp = dequeue();

    if (temp != -1)
    {
        printf("Dequeued Value : %d\n", temp);
    }
    display();
    break;

case 3:
    temp = peek();

    if (temp != -1)
    {
        printf("Highest Priority Front Element : %d\n", temp);
    }
    display();
    break;

default:
    printf("Program terminating ... \n");
    return 0;
}
}
}

```


OUTPUT :

```
-----
Priority Queue (2D Array)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 1
Enter Value : 27
Enter Priority : 3
Enqueued Successfully.

--- Current Queue State ---
Priority 3 : [27]
Priority 2 : [58]
Priority 0 : [64]
-----

-----
Priority Queue (2D Array)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 3
Highest Priority Front Element : 27

--- Current Queue State ---
Priority 3 : [27]
Priority 2 : [58]
Priority 0 : [64]
-----

-----
Priority Queue (2D Array)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 2
Dequeued Value : 27

--- Current Queue State ---
Priority 2 : [58]
Priority 0 : [64]
-----

-----
Priority Queue (2D Array)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 2
Dequeued Value : 58

--- Current Queue State ---
Priority 0 : [64]
-----

-----
Priority Queue (2D Array)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 2
Dequeued Value : 64
Queue Empty

-----
Priority Queue (2D Array)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 2
Dequeued Value : 64
Queue Empty
```

d. Multiple single linked lists.

SOURCE CODE :

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <stdbool.h>

#define PRIORITY 10

typedef struct Node {
    int val;
    struct Node *next;
} Node;
Node *front[PRIORITY];
void init() {
    for(int i = 0; i < PRIORITY; i++)
        front[i] = NULL;
}
bool isEmpty() {
    for(int i = 0; i < PRIORITY; i++)
        if(front[i] != NULL)
            return false;

    return true;
}
void enqueue(int val, int priority) {
    if (priority < 0 || priority >= PRIORITY) {
        printf("Invalid Priority Level\n");
        return;
    }

    Node *newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("Overflow (Memory Allocation Failed)\n");
        return;
    }

    newNode->val = val;
    newNode->next = NULL;
    if(front[priority] == NULL) {
        front[priority] = newNode;
    } else {
        Node *ptr = front[priority];

        while(ptr->next)
            ptr = ptr->next;

        ptr->next = newNode;
    }

    printf("Enqueued Successfully.\n");
}

int peek() {
```

```

    if(isEmpty()) {
        printf("Underflow\n");
        return -1;
    }

    for(int i = PRIORITY - 1; i >= 0; i--) {
        if(front[i] != NULL)
            return front[i]->val;
    }
    return -1;
}

int dequeue()
{
    if(isEmpty()) {
        printf("Underflow\n");
        return -1;
    }

    for(int i = PRIORITY - 1; i >= 0; i--) {
        if(front[i] != NULL) {
            Node *temp = front[i];
            int val = temp->val;
            front[i] = front[i]->next;
            free(temp);
            return val;
        }
    }
    return -1;
}

void display() {
    if(isEmpty()) {
        printf("Queue Empty\n");
        return;
    }

    printf("\n--- Current Queue State ---");
    for(int i = PRIORITY - 1; i >= 0; i--) {
        if (front[i] != NULL) {
            printf("\nPriority %d : ", i);
            Node *ptr = front[i];
            while(ptr) {
                printf("[%d] -> ", ptr->val);
                ptr = ptr->next;
            }
            printf("NULL");
        }
    }
    printf("\n-----\n");
}

int main()
{

```

```

srand(time(NULL));
init();
int ch, val, priority, temp;
while (1) {
    printf("\n-----\n");
    printf(" Priority Queue (Array of Lists)\n");
    printf("-----\n");
    printf("1. Enqueue\n");
    printf("2. Dequeue\n");
    printf("3. Peek\n");
    printf("4. Exit\n");
    printf(">>> ");
    scanf("%d", &ch);

    switch (ch) {
    case 1:
        val = rand() % 100;
        priority = rand() % 5;
        printf("Enter Value : %d\n", val);
        printf("Enter Priority : %d\n", priority);
        enqueue(val, priority);
        display();
        break;

    case 2:
        temp = dequeue();

        if (temp != -1) {
            printf("Dequeued Value : %d\n", temp);
        }
        display();
        break;

    case 3:
        temp = peek();
        if (temp != -1) {
            printf("Highest Priority Front Element : %d\n", temp);
        }
        display();
        break;

    default:
        printf("Program terminating ... \n");
        return 0;
    }
}
}

```

OUTPUT :

```

-----
Priority Queue (Array of Lists)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 1
Enter Value : 18
Enter Priority : 0
Enqueued Successfully.

--- Current Queue State ---
Priority 0 : [18] -> NULL
-----

-----
Priority Queue (Array of Lists)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 1
Enter Value : 98
Enter Priority : 0
Enqueued Successfully.

--- Current Queue State ---
Priority 0 : [18] -> [98] -> NULL
-----

-----
Priority Queue (Array of Lists)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 3
Highest Priority Front Element : 31

--- Current Queue State ---
Priority 1 : [31] -> NULL
Priority 0 : [18] -> [98] -> [22] -> NULL
-----

-----
Priority Queue (Array of Lists)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 2
Dequeued Value : 31

--- Current Queue State ---
Priority 0 : [18] -> [98] -> [22] -> NULL
-----

-----
Priority Queue (Array of Lists)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 1
Enter Value : 31
Enter Priority : 1
Enqueued Successfully.

--- Current Queue State ---
Priority 1 : [31] -> NULL
Priority 0 : [18] -> [98] -> NULL
-----

-----
Priority Queue (Array of Lists)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 1
Enter Value : 22
Enter Priority : 0
Enqueued Successfully.

--- Current Queue State ---
Priority 1 : [31] -> NULL
Priority 0 : [18] -> [98] -> [22] -> NULL
-----

-----
Priority Queue (Array of Lists)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 2
Dequeued Value : 18

--- Current Queue State ---
Priority 0 : [98] -> [22] -> NULL
-----

-----
Priority Queue (Array of Lists)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 2
Dequeued Value : 98

--- Current Queue State ---
Priority 0 : [22] -> NULL
-----

-----
Priority Queue (Array of Lists)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 2
Dequeued Value : 31

--- Current Queue State ---
Priority 0 : [18] -> [98] -> [22] -> NULL
-----

-----
Priority Queue (Array of Lists)
-----
1. Enqueue
2. Dequeue
3. Peek
4. Exit
>>> 4
Program terminating ...

```